

OBFUSKATION

Entwicklerdokumentation

Aufbau, Bauen und offene Befunde

Inhalt

1. Überblick und Entwurfsentscheidungen

2. Verzeichnisplan und tragende Klassen

Warum die Nutzungsdaten im Index stehen, nicht im Profil

Das kritische Detail beim Umbenennen

3. Datenfluss eines obfuscate-Laufs

4. Das Ableitungsverfahren

5. Schutzmechanismen

6. Erweitern

Neuen Generator hinzufügen

Ausdrucksgenerator (GeneratorExpression/ExpressionGenerator)

Neue Textregel

Neues Dateiformat

Profilversion anheben

7. Die Oberfläche

8. Bauen, testen, freigeben

Eine Falle der Einzeldatei: Paketfassungen aus dem Framework

9. Konfigurationsschema

Erweiterungsdatei

Mustererkennung (ValueSuggester)

Spaltennamen-Vorschlag (FieldNameSuggester)

10. Rückgabewerte der CLI

Profilverwaltung auf der Kommandozeile

Fassung und Ersteller in der Hilfe

11. Befunde

Fassung 1.10.0 · Stand 26. September 2026

Diese Dokumentation richtet sich an alle, die Obfuskation bauen, erweitern oder abnehmen wollen. Sie setzt Vertrautheit mit C# und .NET voraus und verweist auf konkrete Dateien und Zeilen, statt Code abzuschreiben.

1. Überblick und Entwurfsentscheidungen

Das Projekt besteht aus drei Projekten:

src/Obfuskation.Core/	Klassenbibliothek – die gesamte Fachlogik
src/Obfuskation.Cli/	Kommandozeilenprogramm, eine dünne Hülle darum
src/Obfuskation.Gui/	Oberfläche (Avalonia), ebenfalls nur eine Hülle
tests/	xUnit – Core, Kommandozeile und Oberfläche getrennt

Die Bibliothek kennt weder Konsole noch Fenster: `ObfuscationEngine` gibt nichts aus, wirft stattdessen typisierte Ausnahmen und liefert Berichte als Datenobjekte (`RunReport`).

`Obfuskation.Cli` bildet das auf Konsolenausgabe und Rückgabewerte ab (`ConsoleOutput` , `ExitCodes`), `Obfuskation.Gui` auf Ansichtsmodelle und Fenster.

Warum diese Trennung: dieselbe Engine und dieselbe Ersetzungstabelle für beide Programme bedeutet, dass ein mit der Oberfläche pseudonymisierter Datensatz sich auf der Kommandozeile zurückholen lässt und umgekehrt — ohne diese Zusage wäre das Werkzeug für den Wechsel zwischen interaktiver Arbeit und Skripten unbrauchbar. Abgesichert wird das durch `tests/Obfuskation.Gui.Tests/EquivalenceTests.cs` : die Tests bauen ein Profil einmal über `ObfuscationEngine` unmittelbar und einmal über `ProfileSession` / `MainViewModel` auf und vergleichen die Ausgabebytes. Am laufenden Programm belegt das Rohprotokoll denselben Befund an echten Dateien (G-08): `diff demo/stammdaten.pseudo.csv arbeit/stammdaten.pseudo.csv` lieferte keine Abweichung — GUI und CLI erzeugten byteweise identische Ausgaben.

2. Verzeichnisplan und tragende Klassen

Klasse / Bereich	Pfad	Aufgabe
ObfuscationEngine	src/Obfuskation.Core/ObfuscationEngine.cs	Einstiegspunkt für Obfuscate, Deobfuscate, Scan und Analyze. Prüft das Profil im Konstruktor und verweigert ein fehlerhaftes sofort.
Profile und Unterklassen	src/Obfuskation.Core/Configuration/Profile.cs	Reines Datenobjekt (Version, Felder, Textregeln, Generatoreinstellungen) ohne Verhalten, damit die Oberfläche es direkt an Formulare binden kann.

Klasse / Bereich	Pfad	Aufgabe
ProfileValidator	src/Obfuskation.Core/Configuration/ProfileValidator.cs	Prüft ein Profil auf Widersprüche und liefert eine Liste von ValidationIssue mit Feldpfad statt einer Sammelmeldung.
MappingStore	src/Obfuskation.Core/Mapping/MappingStore.cs	Verwaltet die Ersetzungstabelle: Laden, Sperren, atomares Schreiben, Rechtevergabe, Rückwärtsindex je Namensraum.
Pseudonymizer	src/Obfuskation.Core/Mapping/Pseudonymizer.cs	Bildet Klartext auf Pseudonym ab und zurück; behandelt Kollisionen und die Sonderfälle nicht umkehrbarer und selbst-umkehrender Generatoren.
SeedDeriver	src/Obfuskation.Core/Generation/SeedDeriver.cs	Leitet deterministische Seeds per HMAC-SHA256 aus dem Profil-Salt ab.
GeneratorRegistry	src/Obfuskation.Core/Generation/GeneratorRegistry.cs	Baut die konfigurierten Generatoren eines Laufs auf, einschließlich eigener Namensräume aus profile.Generators.
TextRuleEngine	src/Obfuskation.Core/Detection/TextRuleEngine.cs	Wendet Textregeln überschneidungsfrei auf Freitext an; alle Regeln laufen gegen den unveränderten Originaltext.
ReverseTextMapper	src/Obfuskation.Core/Detection/ReverseTextMapper.cs	Führt beliebigen Text auf Echtwerte zurück, längster Treffer zuerst, mit Wortgrenzen bei wortartigen Pseudonymen.
GeneratorDescriptions	src/Obfuskation.Core/Generation/GeneratorDescriptions.cs	Deutsche Erklärungen zu jedem eingebauten Generator, gemeinsam genutzt von Oberfläche und Hilfetexten.
Prozessoren	src/Obfuskation.Core/Formats/{Csv,Json,PlainText}Processor.cs	Zerlegen und Zusammensetzen der drei Dateiformate; delegieren die Feldentscheidung an einen IRecordTransformer.

Klasse / Bereich	Pfad	Aufgabe
Transformer	src/Obfuscation.Core/Formats/{Obfuscate, Deobfuscate, Scan}Transformer.cs	Implementieren IRecordTransformer und damit die eigentliche Fachlogik je Vorgang, unabhängig vom Dateiformat.
PathHelper	src/Obfuscation.Core/PathHelper.cs	Pfadauflösung an einer Stelle: ConfigDirectory, ProfileDirectory, DefaultProfilePath, DefaultMappingStorePath und ResolveMappingStore — Letztere gemeinsam genutzt von ObfuscationEngine.ResolveMappingStorePath und ProfileCatalog, damit die Regel nicht zweimal dasteht.
ProfileIndex	src/Obfuscation.Core/Configuration/ProfileIndex.cs	Der Nutzungs-Index: merkt sich Pfade bearbeiteter Datendateien je Profil und wann ein Profil zuletzt benutzt wurde. Getrennt von der Profildatei selbst — Begründung unten.
ProfileCatalog	src/Obfuscation.Core/Configuration/ProfileCatalog.cs	Sammelt alle bekannten Profile (zentraler Ordner plus Zusatzpfade) zu ProfileSummary-Zeilen für Übersicht und profile list; ein unlesbares Profil ergibt einen Eintrag mit gesetztem Error, nie eine Ausnahme.
ProfileRenamer	src/Obfuscation.Core/Configuration/ProfileRenamer.cs	Der sicherheitskritische Teil der Profilverwaltung: benennt ein Profil um, ohne den Bezug zu seiner Ersetzungstabelle zu verlieren (Einzelheiten unten).

Warum die Nutzungsdaten im Index stehen, nicht im Profil

Naheliegender wäre gewesen, Pfad und Zeitpunkt der zuletzt bearbeiteten Dateien gleich im Profil selbst mitzuführen — ein Feld `LastUsedFiles` neben `fields` und `textRules`. Das wurde bewusst nicht so gebaut, aus einem einzigen, aber durchschlagenden Grund: die Profildatei ist die Regeldatei, und die Regeldatei wird **nie ungefragt geschrieben**. Jede Feldregeländerung in der Oberfläche lebt bis zum ausdrücklichen Klick auf „Speichern“ nur im Speicher (`ProfileSession.HasUnsavedChanges`); genau das ist die Grundlage für die Rückfrage bei ungespeicherten Änderungen (Abschnitt 7). Würde das Öffnen einer Datendatei automatisch das Profil auf der Platte aktualisieren, um den neuen Dateipfad zu vermerken, geschähe damit zwangsläufig eines von zwei unerwünschten Dingen: entweder es würde nur der Nutzungs-Teil geschrieben und der Rest der noch unbestätigten Feldregeln im Speicher ignoriert (eine stille Sonderregel, die schwer nachvollziehbar wäre), oder es würden gleich alle unbestätigten Regeländerungen mitgeschrieben — und genau das darf nicht passieren, weil der Anwender sie noch nicht bestätigt hat. Ein einfaches „Datei geöffnet“ hätte damit den Nebeneffekt, ein halbfertiges Regelwerk festzuschreiben.

Die Trennung in eine eigene, ausschließlich maschinell gepflegte Indexdatei (`profil-index.json`) löst das sauber: `ProfileIndex.RecordDataFile` und `RecordProfileUse` schreiben bei jedem Öffnen unbekümmert und ohne Rückfrage, weil dort ausschließlich Pfade und Zeitstempel stehen — nie eine Feldregel, nie ein unbestätigter Zustand. Der Preis dafür ist, dass die Indexdatei rein informativ ist und beim Löschen (`ProfileIndex.Forget` , „Aus Liste entfernen“ in der Übersicht) nichts an den eigentlichen Profilen oder Ersetzungstabellen ändert — genau das macht sie aber auch gefahrlos löschtbar.

Der Fehler hinter „Aus Liste entfernen“ (behooben in 1.4.0): `ProfileCatalog.Collect` (`src/Obfuskation.Core/Configuration/ProfileCatalog.cs:33`) zählt den zentralen Profilordner bei jedem Aufruf vollständig neu auf — richtig so, sonst müsste jedes neu angelegte Profil erst irgendwo eingetragen werden, um in der Übersicht aufzutauchen. `ProfilesViewModel.Remove` rief bislang nur `ProfileIndex.Forget` und entfernte den Pfad aus `GuiSettings.RecentProfiles` — beides betrifft aber nur die *Zusatzpfade* aus dem zweiten und dritten Argument von `Collect` , nie die Verzeichnis-Enumeration selbst. Lag die Profildatei im zentralen Ordner, kam sie beim nächsten `Refresh()` also unweigerlich zurück, nur ohne Nutzungsdaten (daher „keine Dateien“, „nie“ und das Abrutschen in der Sortierung „Zuletzt benutzt“). Die Behebung fügt eine dritte, vom Katalog unabhängige Sperrliste hinzu: `GuiSettings.HiddenProfiles` (`src/Obfuskation.Gui/Services/GuiSettings.cs`), Vollpfade, Vergleich wie bei `RecentProfiles` immer über `Path.GetFullPath` und `StringComparison.Ordinal` . `ProfilesViewModel.Refresh` filtert die Ausgabe von `ProfileCatalog.Collect` gegen `GuiSettings.IsHidden` , bevor sortiert und angezeigt wird — der Katalog selbst weiß nichts von Ausblendungen, das bleibt Sache des Ansichtsmodells, genau wie Sortierung und Filter schon vorher. `BrowseAsync` (Schaltfläche „Aus Datei wählen...“) ruft `UnhideProfile` auf den gewählten Pfad, sonst wäre ein einmal ausgeblendetes Profil auf Dauer unerreichbar. Neu daneben: **„Profil löschen...“** (`ProfilesViewModel.DeleteCommand`) löscht die Profildatei tatsächlich von der Platte, wahlweise samt Ersetzungstabelle und ihrer `.lock` -Datei (`DeleteChoice.ProfileAndMapping`); die Rückfrage dafür läuft über `IDialogService.AskDeleteProfileAsync` und sagt ausdrücklich, dass mit der Tabelle sämtliche Echtwerte verloren gehen. Verweigert wird der Löschvorgang, wenn die Zeile das Profil der laufenden Sitzung ist (`IsCurrentSession`) — sonst arbeitete das Hauptfenster ungebremst mit einer bereits gelöschten Datei weiter.

Das kritische Detail beim Umbenennen

`ProfileRenamer.Rename` (`src/Obfuskation.Core/Configuration/ProfileRenamer.cs:29`) ändert scheinbar nur ein Textfeld, `Profile.ProfileName` . Der Name bestimmt aber über `PathHelper.DefaultMappingStorePath` auch den *Vorgabepfad* der Ersetzungstabelle, solange `Profile.MappingStore` leer ist — und dieser Vorgabepfad wird beim Anlegen nicht laufend neu berechnet, sondern beim allerersten Lauf einmal aufgelöst und ist von da an implizit an den damaligen Namen gebunden. Würde `Rename` den Namen zuerst ändern, zeigte die Vorgabe für den *neuen* Namen auf ein anderes, leeres Verzeichnis: aus Sicht der nächsten Läufe gäbe es plötzlich keine einzige bekannte Pseudonymisierung mehr, und `Zurückholen` fände nichts. Deshalb schreibt `Rename` als ersten Schritt den bisherigen, aus dem *alten* Namen aufgelösten

Pfad ausdrücklich in `Profile.MappingStore` fest (`MappingStorePinned` im `RenameOutcome`), bevor der Name sich ändert. Erst danach folgt der zweite Schritt: existiert die Tabelle bereits, wird der Profilname auch **in ihr** nachgezogen (`MappingStore.RenameProfile`) — ohne diesen Schritt würde `MappingStore.Open` bei jedem folgenden Lauf mit „Der Mapping-Store gehört zum Profil ...“ warnen, weil der im Dokument hinterlegte Name dann vom Profilnamen abweiche. Beide Schritte zusammen sind der Grund, warum Umbenennen überhaupt eigenen Code braucht statt eines einfachen Feldzugriffs; abgesichert durch `ProfileRenameTests`, die wichtigsten Tests des gesamten Vorhabens.

3. Datenfluss eines `obfuscate`-Laufs

1. `ObfuscationEngine`-Konstruktor ruft `ProfileValidator.Validate` auf; bei mindestens einem Befund mit `ValidationSeverity.Error` wirft er sofort `ConfigurationException` — vor jeder weiteren Aktion.
2. `Obfuscate` öffnet den `MappingStore` (`OpenStore`), das legt bei Bedarf Verzeichnis und Datei an, prüft auf ein Git-Arbeitsverzeichnis, setzt die Dateirechte durch und sperrt die Tabelle gegen parallele Läufe.
3. `SeedDeriver`, `GeneratorRegistry` und `Pseudonymizer` werden aus dem Salt des geöffneten Stores aufgebaut.
4. `CreateResolver` baut den `FieldRuleResolver`; im strengen Modus (`--strict`) arbeitet er auf einer Kopie des Profils mit `UnknownField = FieldAction.Error`, damit das geladene Profil selbst unverändert bleibt.
5. Der passende Prozessor (`CsvProcessor`, `JsonProcessor` oder `PlainTextProcessor`) liest das Byte-Array, erkennt Zeichensatz und bei CSV das Trennzeichen (`TextFormatDetector`), und ruft `transformer.OnFields(...)` mit der vollständigen Feldliste auf.
6. **Hier, in `ObfuscateTransformer.OnFields`, erfolgt der Abbruch bei offenen Feldern — vor jeder Verarbeitung eines einzelnen Wertes.** Die Methode sammelt zuerst alle unbehandelten Felder und wirft danach in einem Schritt `UnhandledFieldException` mit der vollständigen Liste. Bis dahin wurde kein Byte der Ausgabe geschrieben und kein Eintrag in die Ersetzungstabelle eingetragen.
7. Erst danach verarbeitet der Prozessor Datensatz für Datensatz; je Feld ruft er `transformer.TransformField(...)` auf, das über den `FieldRuleResolver` die passende Regel bestimmt und je nach Aktion durchreicht, schwärzt, entfernt, mit Textregeln durchsucht oder über den `Pseudonymizer` ersetzt.
8. Bei großen Dateien meldet ein `ProgressReporter` den Fortschritt in Blöcken von 500 Datensätzen; ein `CancellationToken` wird je Datensatz geprüft, damit ein Abbruch nicht erst am Ende einer sehr großen Datei wirkt.
9. Ist der Lauf kein Probelauf, schreibt `store.Save()` die Tabelle atomar (erst in eine Nebendatei, dann Umbenennen). Der `RunReport` wird mit Zählern, Feldbehandlungen und Dauer vervollständigt und zurückgegeben.

`Analyze` (für die Voransicht der Oberfläche) durchläuft nur Schritt 1 und liest die Struktur über `FieldInspector`, ohne Werte zu verarbeiten und ohne bei offenen Feldern abubrechen — die Oberfläche muss den offenen Zustand ja gerade anzeigen können.

4. Das Ableitungsverfahren

Jedes Pseudonym wird deterministisch abgeleitet:

```
seed = HMAC-SHA256(Salt des Profils, Generatorname + Klartext + Zähler)
```

(`SeedDeriver.Derive`, `src/Obfuskation.Core/Generation/SeedDeriver.cs:31`). Die Bestandteile werden längenpräfigiert verkettet, damit `("ab", "c")` und `("a", "bc")` nicht denselben Seed ergeben. Aus der Determinismus-Zusage folgt: derselbe Klartext bekommt im selben Profil über alle Dateien und Läufe hinweg dasselbe Pseudonym — das ist es, was Verknüpfungen über Kundennummern intakt hält.

Kollisionsprüfung in beide Richtungen (`Pseudonymizer.Pseudonymize`, `src/Obfuskation.Core/Mapping/Pseudonymizer.cs:57`): beim Eintragen eines neuen Pseudonyms wird geprüft, dass der Kandidat weder bereits als Pseudonym vergeben ist (`IsPseudonymTaken`) noch selbst als Klartext im Bestand steht (`IsPlaintextKnown`) noch mit dem Original identisch ist. Schlägt eine der drei Prüfungen an, wird der Zähler erhöht und neu abgeleitet, bis zu 100 Versuche (`MaxCollisionRetries`); danach wirft der Pseudonymizer `MappingConflictException` mit dem Hinweis, einen Generator mit größerem Wertevorrat zu wählen.

Warum `dateShift` keinen Tabelleneintrag bekommt: ein verschobenes Datum kann zufällig mit einem echten Datum desselben Bestands zusammenfallen; ein Tabelleneintrag wäre dann mehrdeutig, weil dasselbe Pseudonym-Datum auf zwei verschiedene Klartexte verweisen müsste. `DateShiftGenerator` markiert sich deshalb über `HasIntrinsicInverse => true` (`src/Obfuskation.Core/Generation/DateShiftGenerator.cs:42`), und `Pseudonymizer.Pseudonymize` überspringt für solche Generatoren die Tabelle vollständig — die Rückrechnung erfolgt stattdessen über den profilweit konstanten, aus dem Salt abgeleiteten Offset (`SetOffsetFrom`).

Folge für Freitext: `ReverseTextMapper` (die Rückabbildung in Fließtext) arbeitet ausschließlich über die Einträge der Ersetzungstabelle (`store.AllReverseEntries()`). Da `dateShift` dort keinen Eintrag hinterlässt, kann ein verschobenes Datum, das in freiem Text auftaucht, nicht zurückgeholt werden — nur in CSV- und JSON-Spalten, wo die Feldregel selbst den Bezug zum Generator liefert. Das Rohprotokoll bestätigt das indirekt: bei der Rückübersetzung der KI-Antwort (P-12) traten keine Datumswerte im Freitext auf, die Zurückrechnung betraf dort ausschließlich Namen, Kontonummern, IBAN, BIC, E-Mail, Telefonnummer und Anschrift.

Das Präfix eines `token`-Generators ist reine Kosmetik am Ergebnis

(`TokenGenerator.Configure`, `src/Obfuskation.Core/Generation/SimpleGenerators.cs`): es wird dem fertigen Wert vorangestellt, nachdem `Generate` ihn berechnet hat, und fließt nicht in `seed` ein. Deshalb bleibt die Rückübersetzung unangetastet — `DeobfuscateTransformer` schlägt den kompletten String inklusive Präfix im Mapping nach, `ReverseTextMapper` baut sein Suchmuster aus den tatsächlich gespeicherten (also bereits präfigierten) Pseudonymen und schützt jeden Eintrag mit `Regex.Escape`.

Das Präfix hängt bewusst am `generators`-Eintrag (also am Namensraum) und nicht am Feldnamen: würde es zur Laufzeit aus dem Spaltennamen abgeleitet, bekäme derselbe Klartext in zwei Dateien mit abweichenden Spaltennamen zwei verschiedene Pseudonyme, und genau die dateiübergreifende Verknüpfung bräche, die Abschnitt 7 der Anwenderdokumentation als Kernnutzen beschreibt. Der Validator erzwingt das indirekt mit, indem `prefix` nur am Basistyp `token` erlaubt ist (`ProfileValidator.ValidateGenerators`) — alle anderen Generatoren tragen das Format ihres Wertes, ein Präfix zerstörte das.

5. Schutzmechanismen

Mechanismus	Umsetzung	Test	Beleg im Rohprotokoll
Rechte 0600 auf der Ersetzungstabelle	<code>FilePermissions.RestrictFile</code> , <code>src/Obfuskation.Core/Mapping/FilePermissions.cs:24</code> ; angewandt in <code>MappingStore.Save</code>	<code>SafetyTests.DieErsetzungstabelle_ist_nur_fuer_den_Eigentuemer lesbare</code>	P-07: stat zeigt <code>-rw-----</code>
Verweigerung im Git-Arbeitsverzeichnis	<code>PathHelper.IsInsideGitWorkingTree</code> und die Prüfung in <code>MappingStore.Open</code> , <code>src/Obfuskation.Core/Mapping/MappingStore.cs:71</code>	<code>SafetyTests.DieTabelle_verweigert_die_Ablage_in_einem_Git_Verzeichnis</code>	N-05: Rückgabewert 5, keine <code>mapping.json</code> im Repository angelegt
Sperrdatei gegen parallele Läufe	<code>MappingStore.AcquireLock</code> öffnet <code><pfad>.lock</code> mit <code>FileShare.None</code> und <code>DeleteOnClose</code> , <code>src/Obfuskation.Core/Mapping/MappingStore.cs:148</code>	<code>SafetyTests.Ein_zweiter_Lauf_kann_die_Tabelle_nicht_gleichzeitig_beschreiben</code>	N-06: zweiter Lauf nach 0,4 s bricht mit Rückgabewert 5 ab, der erste läuft unbeeinträchtigt zu Ende
Bericht ohne Werte	<code>RunReport</code> (<code>src/Obfuskation.Core/Reporting/RunReport.cs</code>) führt ausschließlich Zähler, Feld- und Regelnamen; <code>ReportFinding</code> trägt bewusst keinen Fundwert	<code>SafetyTests.Der_Bericht_enthalt_keinen_einzigen_Eingabewert</code>	P-10: der JSON-Bericht von obfuscate enthält Feldnamen und Zähler, aber keinen einzigen Eingabewert

Ergänzend, ebenfalls in `SafetyTests.cs` festgehalten: der Abbruch bei offenen Feldern erfolgt nachweislich vor jeder Verarbeitung (`Der_Abbruch_erfolgt_bevor_irgendetwas_verarbeitet_wurde`), ein Probelauf verändert die Tabelle nicht (`Der_Probelauf_veraendert_die_Ersetzungstabelle_nicht`, im Rohprotokoll P-09 mit `trocken.csv`, das nicht angelegt wurde), und eine in sich widersprüchliche Tabelle — dasselbe Pseudonym auf zwei Klartexte — wird beim Laden zurückgewiesen (`MappingStore.BuildReverseIndex`, `src/Obfuskation.Core/Mapping/MappingStore.cs:174`).

6. Erweitern

Neuen Generator hinzufügen

1. `IPseudonymGenerator` implementieren (`Name`, `IsReversible`, `IsWordLike`, `Generate`; optional `HasIntrinsicInverse` / `TryInvert` und `Configure`) — siehe `src/Obfuskation.Core/Generation/SimpleGenerators.cs` für einfache Beispiele.
2. Die Instanz in `GeneratorRegistry.CreateDefaults()` (`src/Obfuskation.Core/Generation/GeneratorRegistry.cs:21`) eintragen.
3. **Pflicht:** einen Eintrag in `GeneratorDescriptions` (`src/Obfuskation.Core/Generation/GeneratorDescriptions.cs:13`) hinzufügen. Ohne ihn schlägt `GeneratorDescriptionTests.Jeder_eingebaute_Generator_hat_eine_Erklärung` fehl — der Test zählt bewusst über `GeneratorRegistry.KnownNames`, damit ein neuer Generator ohne deutsche Erklärung nicht unbemerkt durchrutscht und in der Oberfläche als nacktes englisches Wort erscheint.
4. Bei Bedarf `ValidateGenerators` in `ProfileValidator.cs` erweitern, falls der neue Generator eigene Pflichteinstellungen mitbringt.

Ausdrucksgenerator (`GeneratorExpression` / `ExpressionGenerator`)

Der Generator `expression` (Plan `docs/plan-ausdruck-generator.md`) erzeugt Werte nach einem RegEx-ähnlichen Ausdruck, aber nur zum Erzeugen, nicht zum Suchen. Die beiden Klassen liegen in `src/Obfuskation.Core/Generation/`:

- `GeneratorExpression` ist der Parser und die Erzeugung. `TryParse` liefert bei einem fehlerhaften Ausdruck `false` und ein `ExpressionError` mit 1-basierter Stelle und deutscher Meldung, statt eine Ausnahme zu werfen — ein fehlerhafter Ausdruck ist eine erwartbare Anwendereingabe (ein Tippfehler im Profil), keine Ausnahmesituation. Intern entsteht ein kleiner Syntaxbaum (Folge, Alternative, Wiederholung, Zeichenklasse, Tabellenverweis, wörtliches Zeichen), rekursiv absteigend geparkt (`GeneratorExpression.Parser`, privat verschachtelt). `{` wird anhand des Folgezeichens unterschieden: eine Ziffer macht es zu einer Anzahl wie `{2,3}`, ein Buchstabe zu einem Tabellenverweis wie `{kreis}`. `ValuePool` schätzt den Wertevorrat (Folge = Produkt, Alternative = Summe, `{n,m}` = $\sum p^k$, Klasse/Tabelle = Größe) und sättigt bei einer übergebenen Obergrenze, damit die Rechnung bei einem sehr

weiten Ausdruck nicht überläuft und für eine reine Warnschwelle nicht genauer sein muss als nötig. `MinLength` / `MaxLength` liefern die kürzeste bzw. längste mögliche Werteslänge, beide von `ProfileValidator.ValidateExpression` genutzt (ein möglicher leerer Wert und eine zu große Obergrenze sind dort Fehler).

- `ExpressionGenerator` implementiert `IPseudonymGenerator`. `Configure` parst den Ausdruck einmal und merkt sich einen Parserfehler statt sofort zu werfen; `Generate` wirft dann erst bei Bedarf eine `GenerationException` (Muster wie bei `WordListGenerator`) — mit fehlendem Ausdruck, Parserfehler oder einer im Ausdruck referenzierten, aber unter `tables` fehlenden Tabelle als Meldung. `IsWordLike` ist `true`, obwohl ein erzeugter Wert kein Wort im üblichen Sinn ist: die Werte sind kurz und unterschiedlich lang (ein Kennzeichen mal mit, mal ohne „E“), und ohne Wortgrenze würde `ReverseTextMapper` einen kürzeren erzeugten Wert auch mitten in einem längeren wiederfinden.
- Die Tabellen (`GeneratorSettings.Tables`, `Dictionary<string, List<string>>?`) liegen im Generator selbst, nicht in einer eigenen Konfigurationsdatei — der Generator ist damit samt seinen Tabellen in sich vollständig und lässt sich unverändert in ein anderes Profil übernehmen. `ProfileStore.JsonOptions` setzt keine `DictionaryKeyPolicy`, Tabellennamen bleiben also nach Speichern und Laden exakt so geschrieben, wie sie eingegeben wurden.

Neue Textregel

Textregeln sind Profildaten (`TextRule`), keine eigenen Klassen. Ein neues Standardmuster gehört in `ProfileScaffolder.DefaultTextRules()` (`src/Obfuskation.Core/Configuration/ProfileScaffolder.cs:139`), mit Priorität, Muster, Ziel-Generator und — falls ein Präfix wie `IBAN:` stehen bleiben soll — `CaptureGroup`. Ein zu weit gefasstes Muster fällt in der Oberfläche im Erprobungsfeld auf (`TextRulesViewModel.Evaluate`); ein Test dafür liegt in `DefaultTextRuleTests.cs`.

Neues Dateiformat

Ein neuer Prozessor implementiert keine eigene Schnittstelle für die Fachlogik — er ruft stattdessen dieselben drei Transformer-Typen (`ObfuscateTransformer`, `DeobfuscateTransformer`, `ScanTransformer`) über `IRecordTransformer` auf (`src/Obfuskation.Core/Formats/IRecordTransformer.cs`). Zu implementieren sind: Struktur einlesen und `OnFields` mit der vollständigen Feldliste aufrufen (bevor irgendetwas geschrieben wird), `ShouldDrop` vor dem Schreiben jedes Feldes befragen, `TransformField` je Wert aufrufen und das Ergebnis übernehmen. `ObfuscationEngine.ResolveFormat` und `RunProcessor` (`src/Obfuskation.Core/ObfuscationEngine.cs:302`) müssen den neuen Fall kennen, ebenso `FieldInspector.Inspect` für die Strukturvorschau ohne Verarbeitung.

Profilversion anheben

`Profile.CurrentVersion` und `MappingDocument.CurrentVersion` (`src/Obfuskation.Core/Configuration/Profile.cs:9` bzw. `src/Obfuskation.Core/Mapping/MappingDocument.cs:15`) sind getrennte Zähler.

`ProfileValidator.Validate` verweigert ein Profil mit höherer Version als Fehler, `MappingStore.Open` ebenso einen Store mit höherer Version als `MappingConflictException`. Ein Versionssprung sollte immer mit einer nachvollziehbaren Migration oder zumindest einer klaren Fehlermeldung einhergehen, welche Version das Programm mindestens braucht.

7. Die Oberfläche

MVVM ohne Framework: `ObservableObject` und `RelayCommand` / `AsyncRelayCommand` (`src/Obfuskation.Gui/ViewModels/ObservableObject.cs`, `RelayCommand.cs`) sind Eigenbau, keine externe Bibliothek. `MainViewModel` kennt weder `Window` noch einen `DateDialog` unmittelbar:

- **Dialoge hinter einer Schnittstelle:** der Konstruktor von `MainViewModel` nimmt `Func<IDialogService> dialogs` entgegen (`src/Obfuskation.Gui/Services/IDialogService.cs`); erst beim tatsächlichen Aufruf entsteht der echte `DialogService` mit Bezug auf das laufende Fenster (`src/Obfuskation.Gui/Services/DialogService.cs`), die einzige Umsetzung der Schnittstelle. Dadurch lässt sich das Ansichtsmodell in Tests mit einer Fabrik füttern, die entweder eine Ausnahme wirft, falls doch ein Dialog aufgerufen würde (`tests/Obfuskation.Gui.Tests/EquivalenceTests.cs:125`), oder — für die Profilverwaltung — mit `FakeDialogService` (`tests/Obfuskation.Gui.Tests/FakeDialogService.cs`) vorgegebene Antworten liefert: welche Wahl bei der Rückfrage „ungespeicherte Änderungen“ getroffen wird, welcher Name und Ablageort beim Anlegen herauskommen, oder welches Profil aus der Übersicht gewählt wird — alles ohne ein einziges echtes Fenster.
- **Nebenfenster als Ereignis** (die drei bestehenden aus Fassung 1.0.0, dazu `HelpRequested` für die Kurzhilfe aus 1.1.0) und **Rückfragen über `IDialogService`** (neu in 1.1.0): `TextRulesRequested`, `MappingRequested`, `AboutRequested` und `HelpRequested` sind einfache `Action`-Ereignisse (`src/Obfuskation.Gui/ViewModels/MainViewModel.cs:78`); `MainWindow.axaml.cs` abonniert sie und öffnet das jeweilige Fenster. Die Kurzhilfe (`HelpWindow.axaml`) folgt damit demselben schlichten Muster wie die drei älteren Nebenfenster, nicht dem `IDialogService` der Profilverwaltung — sie liefert kein Ergebnis, auf das der Aufrufer warten müsste, sondern zeigt nur an. Die drei neuen Fenster der Profilverwaltung — `NewProfileWindow`, `RenameProfileWindow`, `ProfilesWindow` (alle unter `src/Obfuskation.Gui/Views/`) — laufen dagegen über `IDialogService`-Methoden, die ein `Task<TErgebnis?>` liefern: `AskNewProfileAsync`, `AskRenameProfileAsync`, `ShowProfilesAsync`. Der Unterschied zu den älteren Nebenfenstern: diese drei brauchen ein Ergebnis, auf das der Aufrufer wartet (den gewählten Namen, das gewählte Profil), ein Ereignis ohne Rückgabewert würde dafür nicht reichen. `ConfirmWindow` (`src/Obfuskation.Gui/Views/ConfirmWindow.axaml`) ist der schlichte, wiederverwendbare Meldungsdialog mit bis zu drei Schaltflächen dahinter — Avalonia bringt keinen mit; er bedient sowohl `AskSaveChangesAsync` als auch die Umbenennen-Rückfrage. Jede Schaltfläche

schließt das Fenster mit ihrem `Tag` als Ergebniswert; ohne Auswahl (etwa Schließen über die Titelleiste) liefert `ShowDialog<string?>` `null`, was der Aufrufer als die vorsichtige Richtung wertet — „Abbrechen“, nie „Verwerfen“.

- Mehrfachauswahl der Feldliste** (neu in 1.2.0): `SelectedItems` einer `ListBox` ist keine bindbare Eigenschaft wie `SelectedItem` — die Ansicht meldet die Auswahl deshalb selbst, über `MainWindow.OnFieldSelectionChanged` an `MainViewModel.UpdateSelection(...)`. Das Ansichtsmodell bleibt damit fensterfrei und prüfbar: die Tests rufen `UpdateSelection` unmittelbar auf. `SelectedField` bleibt daneben bestehen und bezeichnet das *führende* Feld der Auswahl — nach ihm richten sich Überschrift und Vorschau; es wechselt nur, wenn es aus der Auswahl herausfällt, sonst spränge die Regelkarte bei jedem Erweitern um. Die Regelkarte bindet Aktion und Generator seit 1.2.0 nicht mehr an `SelectedField...`, sondern an `MainViewModel.SelectedAction` und `.SelectedGenerator`: gelesen wird am führenden Feld, geschrieben über `ApplyToSelection` auf alle gewählten. Während einer solchen Zuweisung unterdrückt ein Merker die Nacharbeit der einzelnen Felder (Profilprüfung, Vorschau, Zähler) und lässt sie einmal am Ende laufen — bei hundert Spalten wäre es sonst hundertmal dieselbe Prüfung.
- Warum es die Kurzhilfe gibt:** `docs/anwenderdokumentation.md` erklärt das Programm vollständig, aber eine Anleitung, die neben dem Programm liegt, wird erfahrungsgemäß nicht gelesen — wer die Oberfläche zum ersten Mal öffnet, öffnet kein zweites Dokument dazu. Ohne einen Hinweis unmittelbar im Programm erschließt sich insbesondere nicht, wozu ein Profil überhaupt gut ist: dass es Feldregeln und Ersetzungstabelle bündelt und deshalb dieselbe Datei bei einem falschen Profil ein anderes Pseudonym bekäme. Die Kurzhilfe (`HelpWindow.axaml`) trägt diese Erklärung deshalb in eine einzige, bewusst kurz gehaltene Ansicht im Programm selbst, mit einem Verweis am Ende auf die ausführliche Anwenderdokumentation für alles, was darüber hinausgeht.
- Schutz vor Datenverlust:** `MainViewModel.EnsureChangesHandledAsync()` (`src/Obfuskation.Gui/ViewModels/MainViewModel.cs`) fragt über `AskSaveChangesAsync` nach, sobald `HasUnsavedChanges` zutrifft, und liefert `false`, wenn der Anwender „Abbrechen“ wählt; aufgerufen vor `LoadProfile` in `OpenProfileAsync`, `NewProfileAsync` und dem Öffnen eines Profils aus der Übersicht (`ShowProfilesAsync`). Beim Fensterschließen greift dieselbe Methode über einen kleinen Umweg: `Window.Closing` kann nicht auf eine `Task` warten, deshalb bricht `App.axaml.cs` den ersten Schließversuch mit `e.Cancel = true` ab, startet `EnsureChangesHandledAsync()` und ruft bei `true` über einen Merker `_closeConfirmed` ein zweites Mal `window.Close()` — der Merker muss vor diesem zweiten Aufruf gesetzt sein, sonst entstünde eine Schleife aus Abbrechen und erneutem Rückfragen.
- Mehrfachauswahl bei „Neu aus Datei...“** (neu in 1.4.0): `IDialogService.OpenDataFilesAsync` ergänzt `OpenDataFileAsync` um einen Picker mit `AllowMultiple = true`; die Einzelfassung bleibt für „Öffnen...“ unverändert bestehen, da dort weiterhin genau eine Datei sinnvoll ist. `ProfileScaffold.Create` bekommt eine zweite Überladung mit `IEnumerable<string> sampleFilePaths`

(`src/Obfuskation.Core/Configuration/ProfileScaffolder.cs`): die Feldnamen aller Dateien werden in Lesereihenfolge vereinigt, ein doppelt auftretender Name (etwa die gemeinsame Schlüsselspalte zweier Tabellen) erscheint nur beim ersten Auftreten. Die Einzelfassung ruft seither diese Überladung mit einem Ein-Element- oder leeren Array — wichtig für Aufrufer: ein Aufruf mit dem Literal `null` als zweitem Argument ist damit mehrdeutig (`string?` und `IEnumerable<string>` passen beide) und braucht einen Cast, siehe `InspectionTests.cs` . `ProfileSession.Create` erhält dieselbe zweite Überladung. `MainViewModel.NewProfileAsync` schlägt den Profilnamen aus der *ersten* gewählten Datei vor, baut das Regelgerüst aus *allen*, trägt nach dem Speichern alle gewählten Dateien über `ProfileIndex.RecordDataFile` ein (damit Schnellwahl und Sammellauf sie sofort kennen) und öffnet zuletzt nur die erste über `LoadDataFileAsync` — die sichtbare Feldliste zeigt also weiterhin nur die Spalten der gerade geöffneten Datei, während `Session.Profile.Fields` bereits alle kennt.

- Sammellaufe (neu in 1.4.0):** `MainViewModel.RunBatchAsync` steht neben dem bestehenden `RunAsync` und treibt `ObfuscateAllCommand` / `DeobfuscateAllCommand` . Die Dateiliste kommt aus `RecentDataFiles` , geprüft über ein frisches `File.Exists` statt über das zwischengespeicherte `RecentFileViewModel.Exists` — Letzteres stammt vom letzten Aufbau der Schnellwahl und würde eine zwischenzeitlich gelöschte Datei nicht erkennen. Die einzige Rückfrage läuft über die neue `IDialogService.AskBatchRunAsync(BatchRunProposal)` (Dateianzahl, Namensmuster, Anzahl der zu überschreibenden Zieldateien); danach folgt keine weitere Unterbrechung. Vor der Rückfrage fallen Dateien heraus, deren Ziel ihr eigener Pfad wäre: `DialogService.SuggestOutputName` hängt einen schon vorhandenen Zusatz bewusst kein zweites Mal an, ein Sammellauf über `kunden.pseudo.csv` schreibe sonst über seine eigene Eingabe. Sie werden in der Abschlussmeldung genannt. Jede Datei wird vollständig gelesen, verarbeitet und erst dann geschrieben — ein Abbruch (`CancellationTokenSource` , wie bei `RunAsync`) wirkt nur *zwischen* zwei Dateien, nie mitten in einer, damit nie eine halbe Ausgabedatei entsteht. Ein Fehler an einer einzelnen Datei (`UnhandledFieldException` , `ConfigurationException` , `MappingConflictException` , `MappingLockedException` , `GenerationException` , `IOException` , `UnauthorizedAccessException`) überspringt nur diese eine Datei; Name und Grund landen in der Abschlussmeldung (`BuildBatchStatusText`), nichts wird still übergangen. Die Einzelberichte fasst `MergeReport` zu einem `RunReport` zusammen: `RowsProcessed` und `NewMappings` werden addiert, `RuleHits` je Schlüssel aufsummiert, `Findings` und `Warnings` aneinandergehängt, `TotalMappings` bleibt der Stand des *letzten* Laufs (die Tabelle wächst monoton, ein Aufsummieren würde sie vervielfachen). `RunReport.Command` steht danach auf `obfuscateAll` beziehungsweise `deobfuscateAll` ; `RunResultViewModel.Headline` übersetzt das in „Pseudodateien erzeugt“ / „Klartextdateien erzeugt“.
- Themen** liegen unter `src/Obfuskation.Gui/Themes/` (`Colors.axaml` , `Controls.axaml` , `Metrics.axaml`); `ThemeService.Apply` (`src/Obfuskation.Gui/Services/ThemeService.cs`) setzt nur `Application.Current.RequestedThemeVariant` , die Farbumschaltung selbst

übernimmt Avalonias `ThemeDictionaries`.

- **Einstellungen (neu in 1.8.0):** `SettingsViewModel` (`src/Obfuskation.Gui/ViewModels/SettingsViewModel.cs`) ersetzt das frühere Fenster „Textregeln“ (`TextRulesWindow` , entfernt) und den rein lesenden Dialog „Hauseigene Muster...“ (`ExtensionsWindow` / `ExtensionsViewModel` , beide entfernt). Es arbeitet auf **Kopien**: `ProfileStore.DeepCopy<T>` (JSON-Rundreise, neu) für das Profil, `ExtensionLibrary.Clone()` (dieselbe Rundreise) für die Erweiterung. „Abbrechen“ verwirft die Kopien einfach. „Übernehmen“ prüft zuerst mit `ProfileValidator.Validate(profileCopy, extensionsCopy)` — ein `ValidationSeverity.Error` blockiert und erscheint als Liste im Fenster — und schreibt bei Erfolg: eine geänderte Erweiterung über `ExtensionLibrary.Save` auf die Platte und per `ExtensionLibrary.ReplaceWith(other)` (neu) in **dieselbe**, von `ProfileSession` , `TextViewModel` und `MainViewModel` geteilte Instanz zurück — nicht in eine neu zugewiesene, die nur eine dieser Stellen erreichte; ein geändertes Profil per Listen-Rücktransfer (`TextRules` / `Generators`) in `ProfileSession.Profile` , gefolgt von `ProfileSession.MarkChanged()` . `MainViewModel.ShowSettingsAsync` erledigt danach, was für beide Fälle gleich ist: `InvalidateEngine()` , `RefreshGenerators()` , `RefreshIssues()` , `Text?.RefreshPreview()` und die Statuszeile. Schließen des Fensters mit ungespeicherten Änderungen (über die Titelleiste, nicht über „Abbrechen“) fragt über ein `ConfirmWindow` mit drei Schaltflächen nach (`DialogService.ShowSettingsAsync` , demselben Closing-Umweg wie `App.axaml.cs` , siehe oben).

Themen statt Ablageort (1.9.0): `SettingsTab` trägt seither die Werte `TextRules` , `Generators` , `FieldRules` , `Location` — die früheren Werte `Project` / `Global` sind entfallen, ein eigener Reiter je Ablageort gibt es nicht mehr. Neu `RuleScope { Project, Global }` : wo eine Regel gilt, steht jetzt an ihr selbst statt am Reiter. `TextRulesViewModel` (`src/Obfuskation.Gui/ViewModels/TextRulesViewModel.cs`) wurde dafür vollständig umgebaut — statt zwei Instanzen (eine je Bereich, mit schreibgeschützt gespiegelten Zeilen des jeweils anderen Bereichs, `IsOtherArea`) gibt es jetzt genau **eine**, die `Profile?` und `ExtensionLibrary` direkt hält: `Rules` enthält zuerst die Projektregeln, dann die globalen, jede als `TextRuleViewModel` mit `Scope` , `IsEditable` (Projekt immer, global nur bei beschreibbarer Erweiterungsdatei), `IsLocked` (global und nicht beschreibbar) und `IsOverridden` (eine globale Regel, deren Name eine Projektregel schon vergeben hat — dieselbe Logik wie `ExtensionLibrary.MergeTextRules`). Eine Filter-ComboBox (`RuleFilterOption` , `SelectedFilterOption`) blendet auf „alle“, „dieses Projekt“ oder „alle Projekte“ ein; ohne Profil bleibt sie verborgen (`ShowFilter`).

Der Bereichswechsel läuft jetzt über die Regel selbst:

`TextRuleViewModel.IsProjectScope` / `IsGlobalScope` (ein `RadioButton`-Paar mit Settern, wie `AlwaysReplaceViewModel.UseProfile/UseExtension`) ruft `TextRulesViewModel.ChangeScope` (vormals `SettingsViewModel.MoveRule` , unverändertes Verhalten: ein eigener Generator, den die Regel nutzt und den es im Ziel noch nicht gibt, wird **kopiert, nicht verschoben**, der Name bleibt über `TextRuleNaming.MakeUnique` eindeutig). Eine gesperrte, noch nicht

überschriebene globale Regel trägt zusätzlich

`CanAdjustForProject` / `AdjustForProjectCommand`: „Für dieses Projekt anpassen“ legt per `ProfileStore.DeepCopy` eine gleichnamige Kopie im Profil an, die die globale Regel damit für dieses Projekt überschreibt, ohne die Erweiterungsdatei anzufassen.

Neu je Regel der **Erfassungsmodus** (`enum PatternMode { Exact, Shape, Custom }`, `IsExactMode` / `IsShapeMode` / `IsCustomMode`): beim Aufbau entscheidet `PatternFromSample.TryRecognize(rule.Pattern)` (neu, siehe unten), ob eine bestehende Regel als „Alles dieser Form“ oder „Genau dieser Wert“ wieder erkannt wird, samt einem passenden Beispielwert; ein leeres Muster startet im Formmodus mit leerem Beispiel, alles andere gilt als „Eigener Ausdruck“ (Custom). Der `Sample`-Setter erzeugt außerhalb des Custom-Modus das Muster neu (Form nur bei `CanUseShape`, sonst wörtlich) und hält, solange der Name automatisch ist (`_nameIsAuto`, nur bei einer frisch über „+ Neue Regel“ angelegten Regel und nur bis zur ersten Handänderung am Namensfeld), den Namen über `PatternFromSample.SuggestRuleName` plus denselben `TextRuleNaming.MakeUnique`-Helfer nach. `PatternDescription` (zweite Zeile der Liste) nutzt dieselbe Erkennung, sonst das gekürzte Muster; `PatternError` / `PatternWarning` melden ein leeres bzw. ungültiges Muster oder einen Treffer auf den leeren Text live im Formular. Ein Wechsel aus Custom zurück nach Exact/Shape bildet das Muster nur neu, wenn schon ein Beispielwert da ist — sonst löscht ein Klick einen eigenen Ausdruck. `IsExactMode` gilt auch bei gewählter Form ohne Ziffern im Beispiel (wie `AlwaysReplaceViewModel.UseLiteral`).

Umbenennen. Die Bezeichnung (`Name`) steht oben im Formular; `NameHint` (über `TextRulesViewModel.DescribeNameConflict`) meldet beim Tippen einen doppelten Namen im selben Bereich bzw. eine gleichnamige Regel im anderen Bereich (Überschreiben nach `MergeTextRules`). Freitextfelder wählen Regeln per Namen aus (`FieldRule.TextRules`); damit ein Umbenennen — von Hand oder beim Bereichswechsel mit `MakeUnique` — diese Auswahl nicht bricht, merkt sich `SettingsViewModel` beim Öffnen je Feld die genannten Namen und je Name das damals gemeinte Regelobjekt (Projekt vor global).

`RewriteFieldRuleReferences` bildet daraus bei „Übernehmen“, **vor** der Prüfung, jedes Mal frisch die heutigen Namen; Zwischenstände beim Tippen (kurzzeitig gleichnamig mit einer anderen Regel) spielen so keine Rolle. Die Feldauswahl wird dann zusammen mit den Regeln ins Profil zurückgeschrieben. Andere Profile, die eine globale Regel namentlich auswählen, kennt das Fenster nicht — dafür steht ein Hinweis im Tooltip.

`AlwaysReplaceViewModel` hat dafür das Feld „Bezeichnung“ (`RuleNameInput`, folgt `SuggestedRuleName` bis zur ersten Eingabe, leer fällt auf den Vorschlag zurück; `RuleNameHint` sagt eine angehängte Nummer vorher). `PatternFromSample.SuggestRuleName` schlägt ohne Buchstaben, aber mit Ziffern `nummer` vor (sonst weiter `begriff`), `Describe` nennt Leerzeichen am Rand eines wörtlichen Stücks „Leerzeichen“ statt „ “.

`Views/TextRulesPanel.axaml` (`UserControl`, Inhalt des früheren `TextRulesWindow.axaml`) bedient jetzt genau einen Reiter des `Views/SettingsWindow.axaml` (vier `TabItem`, `SelectedTabIndex` als direktes Casting auf `SettingsTab`, keine feste Höhe/ `MaxHeight`

mehr — Ursache des in der Messung gefundenen abgeschnittenen Formulars). Neu `Views/PatternHelpButton.axaml(.cs)` : ein „?“-Knopf mit Flyout (Spickzettel, KI-Hinweis, „Anfrage für die KI kopieren“ über `TopLevel.GetTopLevel(this)?.Clipboard`), `StyledProperty<bool> IsFieldNameContext` unterscheidet die Beispiele für Textregeln von denen für Spaltennamen; der Flyout-Inhalt bindet mit `x:CompileBindings="False"` über `#Root` auf diese Eigenschaft, weil sein eigentlicher `DataContext` die jeweilige Regel ist, nicht der Knopf selbst.

`AlwaysReplaceViewModel.EditManuallyCommand` heißt weiter „In den Einstellungen bearbeiten...“, das Ereignis `EditRulesRequested` ist jetzt aber **parameterlos** — der Reiter selbst kennt keine Ablageorte mehr; `MainViewModel.ShowAlwaysReplaceAsync` liest stattdessen `UseExtension` direkt und übergibt den passenden `RuleScope` als Filter-Vorgabe an `ShowSettingsAsync(SettingsTab.TextRules, scope: ...)` . Dieselbe erweiterte Signatur (`SettingsTab tab, string? ruleName, RuleScope? scope`) bedient `TextViewModel.EditRuleCommand / EditRulesCommand` aus der Fundleiste der Textansicht (`onEditRuleRequested` , analog zu `onRemoveRuleRequested`) — `MainViewModel.EditTextRuleAsync` bestimmt den Bereich selbst aus dem Profilbestand: bei Namensgleichheit gewinnt das Projekt, weil dessen Fassung tatsächlich greift.

Ersetzungstabelle bearbeitbar: `MappingViewModel` bekommt neu `Func<IDialogService> dialogs` und `Action onChanged` . Werte bleiben verdeckt (`ValuesVisible` , immer `false` beim Öffnen); eingeblendet füllt `RefreshEntries()` `Entries` gefiltert über `SearchText` (Klartext und Pseudonym, `OrdinalIgnoreCase`) und `SelectedNamespaceFilter` . Löschen (einzeln über Mehrfachauswahl oder ganze Namensräume über „Leeren...“) läuft über `IDialogService.AskRemoveMappingEntriesAsync(count, namespaceName)` , dann `MappingStore.Open(readOnly: false)` , die neuen Methoden `MappingStore.Remove(namespaceName, plaintext)` und `MappingStore.RemoveNamespace(namespaceName)` , `Save()` , `Dispose()` , ein erneutes `Load()` und der `onChanged` -Aufruf (`MainViewModel : RefreshMappingSummary()` , `Text?.RefreshPreview()`). Eine `MappingLockedException` dabei wird mit „Die Tabelle ist gerade durch einen Lauf gesperrt“ gemeldet statt weitergereicht. Mehrfachauswahl einer `ListBox` bindet sich wie bei der Feldliste nicht direkt (`MappingWindow.OnEntrySelectionChanged` → `MappingViewModel.UpdateSelection`). Pseudonyme bleiben unveränderlich — nur `Remove` / `RemoveNamespace` , kein Bearbeiten: ein geändertes Pseudonym hätte mit bereits erzeugten Pseudodateien nichts mehr zu tun.

Regeln & Generatoren (1.10.0): Das Fenster „Einstellungen“ heißt jetzt „Regeln & Generatoren“ (`SettingsViewModel.WindowTitle` , neu: „Regeln & Generatoren – {Projekt}“ bzw. ohne Profil „– alle Projekte“) — die Kennungen im Code (`SettingsViewModel` , `SettingsTab` , `ShowSettingsCommand` ...) bleiben unverändert, es ändern sich nur sichtbare Texte. Vier gleichwertige Einstiege führen jetzt dorthin: die Kopfzeile (Knopf und Strg+, wie zuvor), „Regeln bearbeiten...“ in Datei- und Textansicht (ersetzt den Knopf „Immer ersetzen...“ der Dateiansicht — der kleine Dialog bleibt nur noch beim Markieren von Text erreichbar) und neu

die Startseite, deren Knopf `StartViewModel.GlobalRulesCommand` auf `MainViewModel.ShowGlobalRulesCommand` zeigt: `MainViewModel.ShowSettingsAsync` bekommt dafür den Parameter `bool globalOnly = false`, der `session: null` statt der laufenden Sitzung an `SettingsViewModel` übergibt — das Fenster zeigt dann unabhängig von einem im Hintergrund geladenen Profil nur die Regeln und Generatoren für alle Projekte (`SettingsViewModel.HasProfile` steuert dafür auch die Hinweiszeile über dem `TabControl` und blendet die Karte „Dieses Projekt“ im Reiter „Eigene Generatoren“ aus). `MainViewModel.ShowAlwaysReplaceCommand` und `CurrentFileContextText()` sind entfallen, da die Dateiansicht den Dialog „Immer ersetzen...“ nicht mehr direkt öffnet.

Herkunft in der Fundliste: `TextViewModel.DetermineScope(TextRule)` bestimmt je Fund über die Regelreferenz, nicht über den Namen, ob sie aus `ProfileSession.Extensions.TextRules` stammt (`RuleScope.Global`) oder aus dem Profil (`RuleScope.Project`) — verlässlich, weil `TextRuleEngine.FindMatches` die Regelreferenz durchreicht und `ExtensionLibrary.MergeTextRules` bei gleichem Namen nur die Profilregel in die Vereinigung aufnimmt. Neu an `TextMatchViewModel : Scope`, `ScopeLabel` („dieses Projekt“/„alle Projekte“), `OriginLabel` (`"{RuleName} · {ScopeLabel}"`), eine zusätzliche Spalte in `TextView.axaml` und `OriginTooltip`, der bei einer der vier eingebauten Regeln erklärt, dass sie beim Anlegen des Projekts aus `ProfileScaffolder.DefaultTextRules` unverändert übernommen wurde.

Projekt-Generatoren wandern mit globalen Regeln:

`ExtensionLibrary.AdoptProjectGenerators(Profile)` geht `TextRules` und `FieldRules` der Erweiterung durch und kopiert per `ProfileStore.DeepCopy` jeden Generator in `Generators`, der weder eingebaut noch `scanText` ist, den es dort noch nicht gibt, aber im übergebenen Profil — Rückgabe die kopierten Namen. Aufgerufen wird sie an zwei Stellen, die beide schon vorher eine globale Regel mit einem nur im Projekt vorhandenen Generator zulassen konnten, ohne ihn mitzunehmen: `SettingsViewModel.Apply` (vor `ProfileValidator.Validate`, kopierte Namen setzen `_globalDirty`) und `AlwaysReplaceViewModel.CreateRule` bei `UseExtension` (vor `_extensions.Save`, mit Rücknahme der kopierten Generatoren im selben `catch`, der auch die Regel zurücknimmt). `TextRuleViewModel.GeneratorScopeHint` bzw. `AlwaysReplaceViewModel.GeneratorScopeHint` zeigen den Hinweis dazu schon vorher unter der Generator-Auswahl. `TextRulesViewModel.ChangeScope` kopierte einen mitgenommenen Generator schon vorher beim Bereichswechsel selbst — das war nicht die Lücke, die `AdoptProjectGenerators` schließt.

Generator-Dialog (`GeneratorEditorViewModel` / `GeneratorEditorWindow`): ein einziges Ansichtsmodell für Anlegen *und* Bearbeiten eines eigenen Generators, aus drei Stellen erreichbar (Dialog „Immer ersetzen...“, Reiter „Eigene Generatoren“, „Neuer Generator...“ neben „Ersetzen durch“ im Regelformular). Es schreibt selbst nichts: der Aufrufer liest `ResultName`, `ResultSettings` und `ResultScope`, nachdem `ApplyCommand` `Confirmed` gesetzt hat — dasselbe Muster wie `AlwaysReplaceViewModel`. Die Optionsfelder

(`ShowPrefix / Prefix` , `ShowPatternMask / Pattern` , ...) spiegeln `FieldRuleViewModel` und richten ihre Sichtbarkeit nach `ProfileValidator.OptionOwnership` ; `BuildResult()` setzt `Type` auf die gewählte `SelectedBaseType` und verwirft dabei alle Optionen anderer Basistypen (ein fremdes `MaxDays` fällt auf die Vorgabe 400 zurück), lässt aber `Formats / Country / Domain` unangetastet, damit von Hand gepflegtes JSON beim Bearbeiten nicht verloren geht. Seit 1.12.0 steht `maxDays` in `OptionOwnership` (Basistyp `dateShift`); als gesetzt gilt es dort nur, wenn der Wert weder 0 noch 400 ist, weil gespeicherte Profile den Vorgabewert an jedem Generator tragen. Beim Bearbeiten eines bestehenden `dateShift` -Generators meldet `MaxDaysChangeWarning` jede Abweichung vom gespeicherten Wert: Die Verschiebung wird aus `maxDays` und dem Salt abgeleitet (`DateShiftGenerator.SetOffsetFrom`), und ohne Tabelleneintrag wären ältere Pseudodaten danach nicht mehr zurückzurechnen. Ist der Generator schon in Verwendung (`users` -Parameter), sperren `CanRename` und `CanChangeType` Name und Art. Die Vorschau läuft über das neue `Obfuskation.Core.Generation.GeneratorPreview.TryExample(key, settings, sample, derive, out example, out error)` : ein Wegwerf-Profil mit genau diesem einen Generator, `GeneratorRegistry.Build` und `Generate` darauf, `ConfigurationException / GenerationException` werden zu `false` samt Meldung. Der `SeedDeriver` dafür lebt eine Dialoglebenszeit lang in `GeneratorEditorViewModel` selbst (`new(SeedDeriver.CreateSalt())`), damit das Beispiel beim Tippen nicht bei jedem Tastenschlag springt. Die Namensprüfung (`NameError`) verbietet leere, unzulässige, eingebaute und im Zielbereich schon vergebene Namen (außer an den Eintrag selbst beim Umbenennen); `NameHint` warnt zusätzlich ohne zu sperren, wenn derselbe Name im jeweils anderen Bereich schon existiert.

Farben im Eingabefeld (`HighlightTextBox`): Ein `TextBox` kann keine Hintergründe je Textabschnitt tragen, darum zeigte die Textansicht ihre Farben bis 1.9.0 nur in der schreibgeschützten Prüffassung. Seit 1.10.0 ist das Eingabefeld ein `HighlightTextBox` : `TextBox` (`src/Obfuskation.Gui/Views/HighlightTextBox.cs` , `StyleKeyOverride => typeof(TextBox)` , damit Vorlage und Stile unverändert greifen). In `OnApplyTemplate` sucht es `PART_TextPresenter` ; in der Fluent-Vorlage von Avalonia 12 liegt der in einem `Panel` innerhalb des `ScrollView`s. Direkt davor setzt es eine eigene Zeichenebene (`TextHighlightLayer`), die damit deckungsgleich unter dem Text liegt und ohne eigenes Zutun mitscrollt. Sie zeichnet je Bereich die Rechtecke aus `presenter.TextLayout.HitTestTextRange(start, length)` , also aus demselben Layout, das den Text setzt (Umbruch, Schrift, Zeilenhöhe stimmen immer), und zeichnet nach jedem `LayoutUpdated` des Presenters neu. Fehlt der erwartete Aufbau (künftige Vorlage), bleibt das Feld ein gewöhnliches Eingabefeld ohne Farben. Die Bereiche liefert `TextViewModel.InputHighlights` (`TextHighlight(Start, Length, Kind)`), gebaut in `RecomputeResultText` parallel zu `InputSegments` ; sie sind nur gültig, solange `InputText` dem Text des letzten Laufs entspricht, sonst ist die Liste leer. Während der Entprellung steht damit keine Farbe an einer verschobenen Stelle. Die beiden Pinsel liegen als statische Felder an `HighlightTextBox` und werden von der Prüffassung (`TextView.Fill`) mitbenutzt.

Verschachtelte Dialoge: `AlwaysReplaceViewModel` und `SettingsViewModel` tragen je die Eigenschaft `Func<GeneratorEditorViewModel, Task<bool>>? ShowGeneratorEditor`. `DialogService.ShowAlwaysReplaceAsync` und `ShowSettingsAsync` setzen sie, bevor sie ihr eigenes Fenster mit `ShowDialog` öffnen, auf `editor => ShowGeneratorEditorAsync(editor, window)` — mit dem gerade entstehenden eigenen Fenster als Besitzer des verschachtelten Dialogs. `IDialogService` selbst bleibt unverändert, das Muster lebt vollständig in `DialogService` und den beiden Ansichtsmodellen; ein Test setzt die Eigenschaft direkt auf einen Handler, der Felder füllt und `ApplyCommand` ausführt, ganz ohne dass dabei ein Fenster entstünde. `TextRulesViewModel` bekommt denselben Anschluss über einen schlichteren Rückruf, `Func<RuleScope, string?, Task<string?>>? createGenerator` (Bereich und Namensvorschlag hinein, neuer Name oder `null` heraus) — `SettingsViewModel.CreateGeneratorForRuleAsync` sperrt den Bereich dabei auf „alle Projekte“, wenn die Regel selbst schon global ist.

`GeneratorEntryViewModel` (Reiter „Eigene Generatoren“) ist seit dieser Fassung nur noch Anzeige: `Name` und `Prefix` haben keine Setter mehr, `DescriptionLabel` (`GeneratorDescriptions.For`) und `OptionsSummary` (eine Kurzfassung wie „Kennzeichnung FW~“ oder „3 Werte“, sonst leer) ersetzen die frühere Inline-Bearbeitung; ein neues `EditCommand` öffnet den Generator-Dialog mit `existingName` und `users`. `SettingsViewModel.EditGeneratorAsync` entfernt bei einer tatsächlichen Umbenennung den alten Schlüssel aus dem Wörterbuch und baut danach beide Generatorenlisten komplett neu, statt den bestehenden Eintrag zu mutieren.

Weil die Bedienlogik so von der laufenden Anwendung entkoppelt ist, lässt sie sich ohne Fenster prüfen — `tests/Obfuskation.Gui.Tests/MainViewModelTests.cs` tut das für Profilwahl, Feldregeln und die drei Vorgänge, sowie neu für die Rückfrage bei ungespeicherten Änderungen und das Anlegen eines Profils; `tests/Obfuskation.Gui.Tests/ProfilesViewModelTests.cs` prüft Sortierung, Filter, das dauerhafte Ausblenden über „Aus Liste entfernen“ samt Rückholen über „Aus Datei wählen...“ sowie „Profil löschen...“ (inklusive der Verweigerung beim Sitzungsprofil), ebenfalls ohne ein echtes Fenster.

`MappingSummary` (`src/Obfuskation.Gui/Services/MappingSummary.cs`) zieht die Logik, die bislang nur `MappingViewModel` kannte — Pfad, Anzahl der Einträge, Existenz der Tabelle —, in eine kleine, eigenständige Hilfsklasse, die jetzt sowohl `MappingViewModel` als auch die neue Untertitelzeile der Kopfzeile (`MainViewModel.ProfileSubtitle`) verwenden. Neu berechnet wird sie nur beim Profilladen und nach einem abgeschlossenen Lauf, nicht bei jeder einzelnen Feldregeländerung — die Tabelle ändert sich schließlich auch nur bei einem tatsächlichen Lauf.

8. Bauen, testen, freigeben

```
dotnet build
dotnet test
```

FISH

Zielframework ist `net8.0` mit `RollForward=Major` in `Directory.Build.props` — auf einem Rechner ohne installierte .NET-8-Laufzeit laufen `dotnet run` und `dotnet test` dadurch trotzdem auf der vorhandenen neueren Laufzeit, während das Zielframework `net8.0` bleibt.

Freigabe über `build-release.sh` (Windows-Gegenstück: `build-release.cmd`, gleiche Optionen):

```
./build-release.sh                # win-x64 UND linux-x64, je beide Programme
./build-release.sh --rid linux-x64 # nur diese Laufzeit
./build-release.sh --cli-only      # ohne Oberfläche
./build-release.sh --gui-only      # nur die Oberfläche
./build-release.sh --self-contained # ohne installiertes .NET lauffähig
./build-release.sh --no-single-file # nicht zu einer Datei zusammenfassen
./build-release.sh --output <pfad> # abweichendes Wurzelverzeichnis
```

FISH

Das Skript leert vor jedem Lauf das Zielverzeichnis `publish/<RID>/`. `dotnet publish` überschreibt nur, was es selbst erzeugt, und lässt alles andere stehen — Reste eines früheren Laufs mit anderen Optionen wären sonst mit ins Auslieferungspaket gewandert. Geleert wird ausschließlich ein Verzeichnis, das genau die Kennung der Ziellaufzeit trägt; `--no-clean` schaltet es ab, etwa um mit `--gui-only` nur die Oberfläche zu erneuern und das Kommandozeilenprogramm daneben stehen zu lassen.

Das Skript veröffentlicht `Obfuskation.Cli` und `Obfuskation.Gui` einzeln statt über die Solution, damit Bibliothek und Testprojekt nicht mitgezogen werden. Es setzt `-p:DebugType=none` (keine `.pdb`-Dateien im Auslieferungsverzeichnis) sowie `-p:PublishSingleFile=true` und `-p:IncludeNativeLibrariesForSelfExtract=true`, damit auch die nativen Bibliotheken der Oberfläche (Skia, HarfBuzz) in der einen Programmdatei landen. Ohne `--self-contained` erwartet das Ergebnis eine installierte .NET-8-Runtime; mit `--self-contained` bringt es sie mit, auf Kosten der Dateigröße. Das Ergebnis liegt unter `publish/<RID>/` und besteht je Laufzeit aus zwei Dateien: `obfuskation` und `obfuskation-gui`.

`-p:DebugType=none` betrifft allerdings nur die eigenen, verwalteten Symboldateien. SkiaSharp und HarfBuzzSharp liefern zu ihren **nativen** Bibliotheken eigene mit — `libSkiaSharp.pdb` allein rund 80 MB —, und die landeten unabhängig davon im Veröffentlichungsverzeichnis. Das Zielprojekt nimmt sie deshalb über das Ziel `SymboldateienNichtVeroeffentlichen` in `src/Obfuskation.Gui/Obfuskation.Gui.csproj` wieder aus `ResolvedFileToPublish` heraus. Ohne das wäre das Windows-Paket dreimal so groß gewesen wie nötig.

Eine Falle der Einzeldatei: Paketfassungen aus dem Framework

Avalonia zieht `System.IO.Pipelines` als NuGet-Paket herein. Für `net8.0` wählt NuGet daraus die Fassung 8.0.0, während die Anwendung über `RollForward=Major` auf einer neueren Laufzeit läuft, deren `System.Text.Json` die Fassung 9.0.0.0 verlangt. Ohne Einzeldatei fällt das nicht auf:

der Host zieht dann die höhere Fassung des Frameworks vor. Als Einzeldatei veröffentlicht liegt jedoch nur noch die mitgepackte 8.0.0 im Bundle und verdeckt die der Laufzeit — jedes Schreiben einer Konfiguration scheiterte damit an

```
Could not load file or assembly 'System.IO.Pipelines, Version=9.0.0.0'
```

Da `MainViewModel.GuardedAsync` die entstehende `FileNotFoundException` als `IOException` abfängt, wurde daraus eine stille Statuszeile statt eines Absturzes: „Neu aus Datei...“ legte kein Profil an und öffnete keine Datei. Behoben in Fassung 1.2.0 durch

```
<PackageReference Include="System.IO.Pipelines" Version="8.0.0" ExcludeAssets="runtime" />
```

in `src/Obfuskation.Gui/Obfuskation.Gui.csproj`: die Bibliothek gehört seit .NET 3 zum Framework, die Paketfassung gehört deshalb nicht in die Ausgabe. Merkposten für weitere Pakete: was auch im Framework steckt, sollte in einer Einzeldatei nicht in einer älteren Fassung mitreisen. Und: die Oberfläche lässt sich für solche Fälle unter `Xvfb` fernsteuern (`xdotool`), womit sich ein Auslieferungsfehler dieser Art ohne echten Bildschirm nachstellen lässt — den Unit-Tests entgeht er, weil er nur im veröffentlichten Ergebnis auftritt.

9. Konfigurationsschema

Aus `src/Obfuskation.Core/Configuration/Profile.cs` und `Enums.cs`.

Profile

Feld	Typ	Vorgabe	Wirkung
version	int	1	Muss $\leq$ der vom Programm unterstützten Version sein, sonst Fehler
profileName	string	"default"	Name des Profils; bestimmt den Standardpfad der Ersetzungstabelle und ist im Store hinterlegt
description	string?	null	Freitext des Anwenders, wofür das Profil da ist. Rein erklärend, ohne Wirkung auf die Verarbeitung; wird von der Oberfläche in Kopfzeile und Profilübersicht angezeigt. Optionales Feld seit Fassung 1.1.0 — <code>Profile.Version</code> bleibt trotzdem 1, da <code>System.Text.Json</code> unbekannte Felder überliest und alte Programme neue Profile ohne diese Angabe lesen können
mappingStore	string?	null	Pfad zur Mapping-Datei, ~ wird aufgelöst; leer heißt <code>~/local/share/obfuskation/<profileName>/mapping.json</code>

Feld	Typ	Vorgabe	Wirkung
input	InputSett ings	siehe unten	Einstellungen zum Einlesen
defaults	ProfileD efaults	siehe unten	Vorgaben für Felder ohne eigene Regel
fields	List<Fie ldRule>	leer	Feldregeln; die erste passende gewinnt
text Rules	List<Tex tRule>	leer	Muster für Freitext und unstrukturierte Dateien
generato rs	Dictiona ry<strin g, Generato rSetting s>	leer	Generatoreinstellungen und eigene Namensräume, adressiert über den Schlüssel

InputSettings

Feld	Typ	Vorgabe	Wirkung
csvDelimiter	string?	null	null = aus der Kopfzeile erkennen; sonst erzwungen
encoding	string?	null	null = erkennen (BOM, dann strenges UTF-8, sonst Windows-1252); sonst erzwungen
hasHeaderRecord	bool	true	Ob die erste CSV-Zeile Spaltennamen enthält; ohne Kopfzeile werden Spalten über ihre Position benannt

ProfileDefaults

Feld	Typ	Vorgabe	Wirkung
unknownField	FieldAction	Error	Behandlung eines Feldes ohne eigene Regel; Passthrough erzeugt eine Warnung bei der Profilprüfung
redactionPlaceholder	string	"***"	Platzhalter für Redact, sofern ein Namensraum keinen eigenen placeholder trägt (siehe GeneratorSettings.Placeholder unten)

Feld	Typ	Vorgabe	Wirkung
empty Value s	List <string>	leer	Werte, die nach Trimmen (ohne Rücksicht auf Groß-/Kleinschreibung) zusätzlich zu "" und reinem Leerraum als faktisch leer gelten, etwa "- " oder "N/A". Solche Werte laufen unverändert durch statt ein Pseudonym zu bekommen, das einen im Original gar nicht vorhandenen Wert vortäuschen würde. Gemeinsamer Helfer <code>ProfileDefaults.IsEffectivelyEmpty</code> , genutzt von Obfuskation, Prüfung und Rückübersetzung gleichermaßen, damit die drei nicht auseinanderlaufen

FieldRule

Feld	Typ	Vorgabe	Wirkung
match	string	""	Muster für den Feldnamen gemäß <code>matchType</code> ; darf nicht leer sein
matchType	FieldMatchType	Exact	Exact (Groß-/Kleinschreibung egal), Regex oder JsonPath (nur JSON)
action	FieldAction	Error	Die Behandlung des Feldes
generator	string?	null	Generatorname, Pflicht bei Pseudonymize
textRules	List<string>?	null	Namen der bei <code>ScanText</code> angewandten Textregeln; leer/null heißt alle
comment	string?	null	Freitext für den Menschen, wird von <code>init</code> gesetzt

TextRule

Feld	Typ	Vorgabe	Wirkung
name	string	""	Eindeutiger Name, muss gesetzt und einmalig sein
priority	int	50	Höhere Werte gewinnen bei überlappenden Treffern
pattern	string	""	Regulärer Ausdruck, muss gültig sein
generator	string	"token"	Generator für Treffer dieser Regel
captureGroup	int	0	Gruppennummer, deren Inhalt ersetzt wird; 0 = gesamter Treffer
ignoreCase	bool	false	Groß-/Kleinschreibung beim Musterabgleich ignorieren

FieldNameRule — nur in der Erweiterungsdatei (`ExtensionLibrary.FieldRules`), nicht im Profil selbst

Feld	Typ	Vorgabe	Wirkung
pattern	string	""	Regulärer Ausdruck für den ganzen Feldnamen; <code>FieldNameSuggester</code> umschließt ihn mit <code>\A(?:...)\z</code> , ein Teiltreffer zählt also nicht
generator	string	""	Generatorname — eingebaut, aus <code>ExtensionLibrary.Generators</code> , oder der Sonderwert <code>scanText</code> für ein Freitextfeld
ignoreCase	bool	true	Anders als bei <code>TextRule.IgnoreCase</code> standardmäßig aktiv: Spaltennamen unterscheiden sich in der Schreibweise ständig, und das frühere eingebaute Namensraten war ebenfalls unabhängig davon
comment	string?	null	Freitext für den Menschen

GeneratorSettings

Feld	Typ	Vorgabe	Wirkung
type	string?	null	Zugrundeliegender eingebauter Generator; leer heißt: wie der Schlüssel selbst — ein anderer Wert erzeugt einen eigenen Namensraum auf Basis dieses Typs
maxDays	int	400	Maximaler Betrag der Datumsverschiebung in Tagen (nur <code>dateShift</code>), höchstens 36 500; 0 wirkt wie die Vorgabe
formats	List<string>?	null	Zusätzlich erkannte Datumsformate (<code>dateShift</code> , <code>dateRange</code> , <code>dateGeneralize</code>), vor den eingebauten Formaten geprüft
country	string?	null	Ländercode für <code>iban/bic</code> , falls sich keiner aus dem Originalwert ableiten lässt
domain	string?	null	Domain für email
from	string?	null	Untere Grenze des Zeitraums für <code>dateRange</code> , ISO-Datum (yyyy-MM-dd); nur zusammen mit <code>to</code> wirksam
to	string?	null	Obere Grenze des Zeitraums für <code>dateRange</code> ; ohne <code>from+to</code> bleibt das Kalenderjahr des Originals erhalten
granularity	string?	null (wirkt wie "month")	Rundungsstufe für <code>dateGeneralize</code> : <code>month</code> , <code>quarter</code> oder <code>year</code>
pattern	string?	null	Zeichenmaske für <code>pattern</code> : A Großbuchstabe, a Kleinbuchstabe, 9 Ziffer, X alphanumerisch, \ escaped das Folgezeichen als wörtlich; ohne Angabe wird die Maske aus dem Original abgeleitet

Feld	Typ	Vorgabe	Wirkung
values	List<string>?	null	Eigene Werteliste für wordList; einzige echte Pflichtoption unter allen Generatoren — ohne sie wirft der Generator eine GenerationException
keepFirst	int	0	Anzahl der am Anfang sichtbar bleibenden Zeichen (nur partialMask)
keepLast	int	0, wirkt wie 4 solange weder keepFirst noch keepLast gesetzt ist	Anzahl der am Ende sichtbar bleibenden Zeichen (nur partialMask); sobald eine der beiden Seiten gesetzt wird, zählt nur noch das Eingestellte
maskChar	string?	null (wirkt wie "*")	Maskierungszeichen für partialMask, muss genau ein Zeichen lang sein
placeholder	string?	null	Eigener Platzhalter, nur für redact; ohne Angabe gilt defaults.redactionPlaceholder. Getrennt von domain, damit ein eigener redact-Namensraum einen abweichenden Platzhalter behalten kann, der nicht von der Profilvergabe überschrieben wird
prefix	string?	null	Kennzeichnung, die jedem erzeugten Pseudonym vorangestellt wird (nur token); Muster <code>^[A-Za-z0-9ÄÖÜäöüß_-]+[~_]\$,</code> höchstens 32 Zeichen — geprüft von ProfileValidator

Welche Option zu welchem Generatortyp gehört, steht nicht in verstreutem Einzelfallcode, sondern in einer einzigen Tabelle: `ProfileValidator.OptionOwnership` (`src/Obfuskation.Core/Configuration/ProfileValidator.cs:50`). Sie ist öffentlich, damit die Oberfläche dieselbe Zuordnung nutzt, um zu entscheiden, welche Optionsfelder sie zu einem gewählten Generator anzeigt — zwei getrennt gepflegte Kopien liefen bei jedem neuen Generator auseinander. Eine Option, die an einem falschen Basistyp hängt (etwa `values` an einem `pattern`-Eintrag), ist ein Validierungsfehler (`ValidationSeverity.Error`), kein stillschweigend wirkungsloses Feld.

Erweiterungsdatei

Zwei feste Fundorte, unabhängig von Kommandozeile oder Oberfläche, geprüft in dieser Reihenfolge — die zuerst gefundene Datei gilt vollständig, es wird nichts zusammengemischt:

```
<Verzeichnis der Programmdatei>/obfuskation.json # ExtensionOrigin.ProgramDirectory
~/.config/obfuskation/obfuskation.json          # ExtensionOrigin.ConfigDirectory,
PathHelper.ConfigDirectory
```

`ExtensionLibrary.ResolvePath(programDirectory = null)` (`src/Obfuskation.Core/Configuration/ExtensionLibrary.cs`) liefert beides: den Pfad, der gelten würde (`null`, wenn keiner der beiden Orte eine Datei hergibt), und alle geprüften Orte samt Befund (`ExtensionCandidate(Path, Origin, Exists, SkippedAsProfile)`) — Grundlage für `extensions path` und `extensions list`, die beide begründen sollen, was sie zeigen, statt ein Ergebnis unerklärt hinzustellen.

Bewusst `Environment.ProcessPath`, nicht `AppContext.BaseDirectory`: bei der Einzeldatei-Veröffentlichung (`PublishSingleFile`, siehe `build-release.sh`) zeigt Letzteres in das Auspackverzeichnis unter `/tmp` und wäre damit für „neben der Programmdatei“ nutzlos. `programDirectory` lässt sich ausdrücklich übergeben — ausschließlich für Tests gedacht, die nicht den echten Pfad des Testläufers ansprechen dürfen.

Schreibregel (`ExtensionLibrary.GetWriteState`, neu in 1.8.0, ersetzt `ResolveWritePath`): liefert einen `ExtensionWriteState(Path, CanWrite, Reason)`. Ohne gefundene Datei gilt `PathHelper.ConfigDirectory` als Ziel, `CanWrite = true`. Eine gefundene, beschreibbare Datei (`IsWritable`, ein kurzer Schreibversuch mit `FileStream / FileAccess.ReadWrite`) bleibt Ziel an ihrem eigenen Ort. Eine gefundene, aber schreibgeschützte Datei liefert `CanWrite = false` mitsamt einer erklärenden `Reason` — und **anders als die frühere** `ResolveWritePath` weicht sie dabei nicht mehr stillschweigend auf `PathHelper.ConfigDirectory` aus. Das war Fehler 2 des Plans zu Fassung 1.8.0: `ResolvePath` liest das Programmverzeichnis weiterhin zuerst (siehe oben); eine über den Konfigurationsordner vorgenommene Änderung an einer Erweiterungsdatei, die eigentlich im Programmverzeichnis liegt und dort schreibgeschützt ist, wäre nach dem nächsten Neustart also wirkungslos gewesen, weil `ResolvePath` weiterhin die (unveränderte) Datei im Programmverzeichnis gefunden hätte. Die Oberfläche sperrt stattdessen den betroffenen Bereich (die globalen Regeln der Einstellungen, siehe Abschnitt 7) und nennt den Grund, statt in eine Datei zu schreiben, die beim nächsten Start ohnehin nicht gilt. Alle Aufrufer wurden auf `GetWriteState` umgestellt (`AlwaysReplaceViewModel.ExtensionPath / CanUseExtension`, `MainViewModel.RemoveTextRuleAsync`, `SettingsViewModel`); `ResolveWritePath` existiert nicht mehr.

Ordnerprüfung (1.9.0): Ist die gefundene Datei selbst beschreibbar, prüft `GetWriteState` zusätzlich das Schreibrecht auf ihren **Ordner** — eine Probedatei `.obfuskation-schreibprobe-<guid>` mit `FileOptions.DeleteOnClose` entscheidet, plattformunabhängig ohne verlässlichere API. Ohne dieses Recht liefert `Save` (Nebendatei, ggf. `.bak`, dann `Move`) sonst erst beim tatsächlichen Speichern einen Fehler, mitten im geöffneten Einstellungsfenster, statt schon beim Öffnen zu sperren.

Abgrenzung zur Profildatei: Die Projektdatei heißt seit Fassung 1.6.0 `obfuskation-projekt.json` (`ProfileStore.DefaultFileName`); der unqualifizierte, frühere Name `obfuskation.json` (`ProfileStore.LegacyFileName`) bleibt als Profil lesbar, sofern er wie eines aussieht. Beide Rollen können also denselben Dateinamen tragen, etwa bei einer tragbaren Ablage, in der Programmdatei und Datenbestand im selben Ordner liegen.

Aufgelöst wird das am Inhalt, mit `ProfileStore.Classify(path)` (neu in 1.9.0, ersetzt das reine Ja/Nein), das drei Werte liefert: `JsonFileKind.Profile` (trägt `profileName` oder `fields`), `.Other` (gültiges JSON, aber kein Profil — etwa eine Erweiterungsdatei) und `.Unreadable` (kein gültiges JSON; `Load` liefert dazu die eigentliche Fehlermeldung). Geparst wird dafür mit denselben `JsonDocumentOptions` wie beim eigentlichen Laden (`CommentHandling = Skip`, `AllowTrailingCommas = true`) — vorher parste `LooksLikeProfile` ohne diese Optionen, eine von Hand kommentierte Erweiterungsdatei löste dabei eine `JsonException` aus, galt damit als „nicht beurteilbar“ und wurde durchgelassen (`true`), also wie ein Profil behandelt und von `ExtensionLibrary.ResolvePath` übersprungen: die hauseigenen Muster wirkten nie, ohne dass irgendetwas das meldete (Fehler 2 der Messung zu 1.9.0, sicherheitsrelevant).

`LooksLikeProfile(path)` bleibt als schmaler Wrapper (`Classify(path)` is `Profile` or `Unreadable`) für `ProfileStore.Discover` (eine gefundene `LegacyFileName` zählt nur als Profil, wenn sie auch wie eines aussieht — sonst bliebe die Suche an der Erweiterungsdatei hängen, statt eine Ebene höher nach einem tatsächlichen Profil weiterzusuchen) und `ProfileCatalog` (siebt fremde Dateien im zentralen Ordner aus) erhalten. `ExtensionLibrary.ResolvePath` ruft dagegen `Classify` **direkt** auf und überspringt nur noch bei `JsonFileKind.Profile` — eine kaputte Datei (`Unreadable`) wird jetzt **gewählt**, und `Load` wirft dafür `ConfigurationException` mit Pfad in der Meldung; die CLI meldet den Konfigurationsfehler-Exitcode (`extensions path` lädt dafür zur Prüfung mit, `extensions list` ohnehin), die Oberfläche sperrt mit der Fehlermeldung. `ExtensionLibrary.Load` fängt zusätzlich `IOException` / `UnauthorizedAccessException` (eine geparste, aber nicht mehr öffnbare Datei) und wickelt auch sie in eine `ConfigurationException`.

Klasse `ExtensionLibrary` (`src/Obfuskation.Core/Configuration/ExtensionLibrary.cs`), dieselben Typen wie im Profil — `Dictionary<string, GeneratorSettings> Generators` und `List<TextRule> TextRules` —, dazu neu `List<FieldNameRule> FieldRules` (siehe Schematabelle oben); kein zweites Schema und keine zweite Validierung.

`ExtensionLibrary.Load(path)` lädt mit denselben `JsonSerializerOptions` wie `ProfileStore` (camelCase, Kommentare und trailing commas erlaubt): eine fehlende Datei ergibt `ExtensionLibrary.Empty`, eine kaputte wirft eine `ConfigurationException` mit Pfad.

Die Erweiterungsdatei wird nie in ein speicherbares `Profile`-Objekt gemischt. Sonst schriebe die Oberfläche beim nächsten Speichern eines Profils die Erweiterungseinträge in jede Profildatei hinein — genau das soll die Trennung verhindern. Stattdessen wird sie an vier Stellen berücksichtigt, mit dem Profil stets als Sieger bei gleichem Schlüssel bzw. Namen:

Ort	Vorrangregel
<code>GeneratorRegistry.Build(profile, deriver, extensions)</code>	Eingebaute Generatoren, dann Erweiterungseinträge, dann Profileinträge — das Profil gewinnt bei gleichem Schlüssel

Ort	Vorrangregel
ObfuscationEngine (Textregeln, über TextRuleEngine)	Erweiterungsregeln plus Profilregeln; bei gleichem name gewinnt die Profilregel, die Erweiterungsregel entfällt dann ganz statt zusätzlich zu greifen
ProfileValidator.Validate(profile, extensions)	Ein generator-Verweis auf einen Erweiterungsschlüssel gilt als bekannt statt als Fehler; Erweiterungseinträge werden mit derselben Logik wie Profileinträge geprüft (OptionOwnership, ValidatePattern, Wertevorrat), Befundpfad extensions.generators.<key> bzw. extensions.textRules[i] bzw. extensions.fieldRules[i]. Verdeckt ein Profileintrag einen gleichnamigen Erweiterungseintrag, ist das nur Info/Warning, kein Error
FieldNameSuggester.Suggest(fieldName, extensions)	Spaltenmuster (FieldRules) fließen nur hier ein, nicht in GeneratorRegistry oder ObfuscationEngine — reines Vorschlagswesen, siehe unten

Kommandozeile: `obfuscation extensions list|path`, gebaut nach demselben Muster wie `mapping list|path` (`BuildMappingCommand`, `src/Obfuscation.Cli/Program.cs`) — mit dem Unterschied, dass `list` hier die Muster selbst zeigt, weil die Erweiterungsdatei anders als die Ersetzungstabelle keine Echtdaten enthält, und zusätzlich nennt, welcher der beiden Fundorte greift. `path` nennt bei Fehlschlag beide geprüften Orte, und bei jedem Ort, an dem eine Datei liegt, aber `SkippedAsProfile` zutrifft, dass dort statt einer Erweiterung ein Profil übergangen wurde. `--no-extensions` als globale Option unterdrückt das Laden für einen einzelnen Lauf.

Oberfläche: Erweiterungseinträge erscheinen in der Generatorauswahl mit Herkunftszusatz. Seit Fassung 1.8.0 lässt sich die Erweiterungsdatei auch ohne Texteditor bearbeiten, in den Reitern „Textregeln“, „Eigene Generatoren“ und „Spalten-Vorschläge“ des Einstellungsfensters (Abschnitt 7) — der Texteditor bleibt weiterhin ein gleichberechtigter Weg, keiner verdrängt den anderen. Lässt sich die Erweiterungsdatei nicht laden, erscheint eine Statuszeile mit dem Pfad, das Programm arbeitet ohne sie weiter, und die globalen Regeln bleiben gesperrt (Fehler 1 des Plans zu 1.8.0: ohne diese Sperre hätte ein „Übernehmen“ die kaputte Datei mit nur den neu angelegten Einträgen überschrieben, statt sie unangetastet zu lassen, bis sie von Hand repariert ist) — eine kaputte Erweiterungsdatei darf die Oberfläche weder blockieren noch überschreiben.

Ein lauffähiges Beispiel liegt unter `docs/beispiel/obfuscation-erweiterung-beispiel.json` — es bildet zugleich das bis 1.5.0 fest einkompilierte Namensraten als `fieldRules` nach (siehe unten).

Testumleitung: Wie beim übrigen Konfigurationszugriff läuft das über `XD_CONFIG_HOME` in `TestUmgebung` für den Fundort im Konfigurationsordner — die Erweiterungsdatei liegt dadurch automatisch im Testverzeichnis. Für den Fundort „neben der Programmdatei“ wird der Pfad im Test ausdrücklich über den `programDirectory`-Parameter von `ResolvePath` übergeben, **nie** aus

`Environment.ProcessPath` des Testläufers abgeleitet — sonst würde ein Test lokal grün und im nächsten Klon (anderer Pfad des Testläufers) rot laufen, oder schlimmer, unbemerkt gegen eine echte Datei neben `dotnet` selbst greifen.

Mustererkennung (`ValueSuggester`)

Klasse `ValueSuggester` (`src/Obfuskation.Core/Configuration/ValueSuggester.cs`), `Suggest(samplesByField, extensions)` liefert je Feld höchstens einen `ValueSuggestion` (Feldname, Generator, Beleg-Beispielwert, Trefferzahl). Verfahren, bewusst streng:

- Kandidatenmuster sind `ProfileScaffolder.DefaultTextRules()` **plus** die Textregeln der Erweiterungsdatei — dadurch schlägt die Erkennung auch Erweiterungsmuster wie `assetTag` vor.
- Ein Muster zählt nur bei **vollständigem** Treffer auf den getrimmten Wert (Regex mit `\A(?:...)\z` umschlossen), nicht bei einem Treffer irgendwo im Wert — ein Teiltreffer ist ein Freitextfall für `scanText`, kein Feldgenerator.
- Leerwerte gemäß `ProfileDefaults.EmptyValues` bleiben außer Betracht, sofern die Vorgaben des Profils übergeben werden (die Oberfläche tut das). Ohne sie zählen nur `""` und reiner Leerraum als leer — beim Gerüstbau ist das richtig, weil ein frisches Profil noch keine eigenen Leerwerte kennt.
- Vorgeschlagen wird nur, wenn **alle** verbleibenden Beispielwerte passen und es mindestens zwei davon gibt. Bei mehreren passenden Mustern gewinnt die höhere `Priority`.
- Dasselbe 5-Sekunden-Timeout und `RegexOptions.CultureInvariant` wie in `TextRuleEngine` — dieselbe Hilfsmethode wird wiederverwendet, nicht neu gebaut.

Spaltennamen-Vorschlag (`FieldNameSuggester`)

Klasse `FieldNameSuggester` (`src/Obfuskation.Core/Configuration/FieldNameSuggester.cs`), `Suggest(fieldName, extensions = null)` liefert einen Generatorknamen, `"scanText"`, oder `null`. Sie ersetzt das frühere, fest einkompilierte `ProfileScaffolder.Hints`-Array (dreißig Paare aus Namensfragment und Generator, per `Contains()` geprüft) durch die `FieldRules` der Erweiterungsdatei — **ersatzlos**, wenn keine Erweiterungsdatei vorliegt oder sie keine `fieldRules` trägt: dann liefert `Suggest` immer `null`, das Programm rät nicht mehr am Spaltennamen. Das ist eine bewusste Verhaltensänderung gegenüber 1.5.0.

Verfahren:

- Jedes `FieldNameRule.Pattern` wird über `TextRuleEngine.Compile` mit `\A(?:{Pattern})\z` umschlossen übersetzt — voller Treffer auf den Feldnamen, kein Teiltreffer; genau das behebt den Fehler der früheren Liste, in der `"ort"` mitten in `Sortiment` traf. Dieselben Optionen und dasselbe 5-Sekunden-Zeitlimit wie bei `TextRuleEngine` und `ValueSuggester` — keine eigene Regex-Kompilierung.
- Die erste passende Regel gewinnt, Reihenfolge in der Datei entscheidet.

- Ein ungültiges Muster wirft hier nichts — das ist Sache von `ProfileValidator`. `FieldNameSuggester` überspringt einen solchen Kandidaten stillschweigend: ein Vorschlag ist eine Erleichterung, kein Prüfschritt.
- Übersetzte Muster werden je `ExtensionLibrary`-Instanz in einer `ConditionalWeakTable` zwischengespeichert (typischerweise einmal pro Lauf geladen und für mehrere Felder wiederverwendet) — kein erneutes Übersetzen bei jedem einzelnen Feldnamen.

Vorrang gegenüber dem Wertetreffer: überall dort, wo ein Vorschlag entsteht, gilt: **ein Wertetreffer (`ValueSuggester`) schlägt eine Spaltennamenregel**, weil ein passender Wert die härtere Aussage ist als ein bloßer Namenstreffer. Findet sich weder ein Wertetreffer noch eine passende `fieldRules`-Regel, bleibt es beim bisherigen Verhalten (eigener `token`-Namensraum mit Kennzeichnung). Die volle Reihenfolge: Wertetreffer → Spaltennamenregel aus der Erweiterungsdatei → eigener `token`-Namensraum.

Genutzt von `ProfileScaffolder.CreateRule` über `obfuskation init --from` (dieselbe Reihenfolge für die Kommentare — die Aktion bleibt `error`, nur der Vorschlagstext wird besser und nennt bei einem Wertetreffer den Beleg-Beispielwert, bei einem Namenstreffer die getroffene Regel) und von `src/Obfuskation.Gui/ViewModels/FieldRuleViewModel.cs` sowie vom Dialog »Muster erkennen...« in der Oberfläche (Anwenderdokumentation, Kapitel 10), der zusätzlich nur Felder vorbelegt, die noch auf `error` stehen.

10. Rückgabewerte der CLI

Aus `src/Obfuskation.Cli/ExitCodes.cs`. Sie sind Teil der Schnittstelle: Skripte werten sie aus, sie ändern sich nicht stillschweigend.

Wert	Bedeutung
0	Erfolg
1	allgemeiner Fehler
2	Konfiguration fehlt oder ist fehlerhaft
3	Feld ohne Regel im strengen Modus
4	scan hat Verdachtsfälle gefunden
5	Ersetzungstabelle widersprüchlich oder gesperrt

Profilverwaltung auf der Kommandozeile

`CommandContext.LoadProfile` (`src/Obfuskation.Cli/CommandContext.cs:18`) nimmt für `--config` seit Fassung 1.1.0 auch einen bloßen Profilnamen an, nicht mehr nur einen Dateipfad: enthält der übergebene Wert kein Verzeichnistrennzeichen und endet nicht auf `.json`, und existiert er nicht wörtlich als Datei, wird zusätzlich unter `PathHelper.DefaultProfilePath`

nachgesehen. Ein Pfad mit Trennzeichen oder mit `.json`-Endung wird weiterhin ausschließlich wörtlich genommen — diese Unterscheidung verhindert, dass eine tatsächlich vorhandene, aber (noch) nicht auffindbare Datei stillschweigend gegen ein gleichnamiges zentrales Profil getauscht wird. Schlägt beides fehl, nennt die Fehlermeldung beide versuchten Orte und verweist auf

`obfuskation profile list`. Abgesichert durch

`tests/Obfuskation.Cli.Tests/CommandContextTests.cs` — dem ersten Testprojekt für die Kommandozeile, mit derselben `TestUmgebung`-Umleitung über

`XDG_CONFIG_HOME` / `XDG_DATA_HOME` wie in den beiden anderen Testprojekten (Befund D-8).

`obfuskation profile list [--sort name|used|changed] [--json] ( Program.cs , BuildProfileCommand )` speist sich aus `ProfileCatalog.Collect(ProfileIndex.Load(), [])` — bewusst ohne Zusatzpfade: die Zuletzt-Liste der Oberfläche (`gui.json`) kennt die Kommandozeile nicht, nur den plattformübergreifenden Nutzungs-Index. Die Ausgabe läuft über zwei neue Methoden in `ConsoleOutput`: `WriteProfiles` (Tabelle auf die Standardfehlerausgabe, wie `WriteSummary`) und `WriteProfilesJson` (auf die Standardausgabe). Sortiert wird in `Program.cs` selbst, nicht in `ProfileCatalog` — dieselbe Aufgabenteilung wie zwischen `ProfileCatalog` und `ProfilesViewModel` auf der Oberflächenseite.

`obfuskation init` kennt zwei neue Optionen: `--description <text>` setzt

`Profile.Description`, `--central` schreibt nach

`PathHelper.DefaultProfilePath(profileName)` statt nach `ProfileStore.DefaultFileName` (`obfuskation-projekt.json`) im aktuellen Verzeichnis — eine ausdrücklich angegebene `--`

`config` gewinnt in jedem Fall. Die Ausgabe nennt seither immer den vollständigen, aufgelösten Zielpfad statt eines möglicherweise relativen.

Fassung und Ersteller in der Hilfe

Unter jeder Hilfeausgabe des Kommandozeilenprogramms — der des Programms selbst wie der jedes Unterbefehls — steht eine einzelne Zeile der Form

```
obfuskation 1.1.0 · Gregor Stübner & Claude (Anthropic)
```

Sie kommt aus `src/Obfuskation.Cli/ProgramInfo.cs`. Die Fassung stammt aus

`AssemblyInformationalVersionAttribute`, also aus `<Version>` in `Directory.Build.props`; der angehängte Commit wird für die Anzeige abgeschnitten. Da `System.CommandLine` keinen Platz für eigenen Text unter der Hilfe vorsieht, umschließt `FooterHelpAction` die vorhandene Hilfeaktion, statt sie zu ersetzen — so bleibt die erzeugte Hilfe unverändert und die Zeile kommt nur hinten dran. Die Oberfläche zeigt dieselbe Angabe im Fenster „Über“ (`src/Obfuskation.Gui/Views/AboutWindow.axaml`).

11. Befunde

Die Befunde D-1 bis D-9 stammen aus der Testdurchführung vom 4. und 5. September 2026 (`docs/bilder/protokoll-roh.md`). N-1 und N-2 kamen am 7. September 2026 aus einer eigenen Nutzungsprüfung hinzu (`befunde/prompt.md` und der beiliegende Screenshot). Keiner der elf betrifft die Richtigkeit der Ersetzung, die Umkehrbarkeit oder den Schutz der Ersetzungstabelle.

Neun der elf Befunde sind inzwischen behoben, einer teilweise, einer bekommt statt einer Codeänderung eine ergänzende Warnung. D-4 und D-8 bereits in Fassung 1.0.1, D-9 in 1.2.0; D-1, D-3, D-5, D-6, N-1 und N-2 in 1.3.0. D-2 bleibt als Eigenschaft bestehen (Abschnitt 4 begründet, warum das so sein muss), bekommt aber seit 1.3.0 eine Warnung im Bericht. D-7 ist seit 1.3.0 nur teilweise behoben, aus Gründen, die beim Eintrag stehen. Behobene Befunde bleiben hier stehen, weil eine Befundliste, aus der Behobenes verschwindet, ihren Wert als Nachweis verliert: sie zeigt dann nicht mehr, was geprüft wurde. Was geändert wurde, steht bei den jeweiligen Einträgen und — für D-4 und D-8 — ausführlich in `docs/testdokumentation.md`, Abschnitt „Behebung“.

Nr.	Art	Stand
D-1	Fehler, kosmetisch	behoben in 1.3.0
D-2	Eigenschaft	Warnung ergänzt in 1.3.0, Eigenschaft bleibt
D-3	Bedienbarkeit	behoben in 1.3.0
D-4	Fehler, zerstörend	behoben in 1.0.1
D-5	Darstellung	behoben in 1.3.0
D-6	Darstellung	behoben in 1.3.0
D-7	Sprache	teilweise behoben in 1.3.0
D-8	Testhygiene	behoben in 1.0.1
D-9	Fehler, Auslieferung	behoben in 1.2.0
N-1	Darstellung	behoben in 1.3.0
N-2	Bedienbarkeit	behoben in 1.3.0

D-1 (Fehler, kosmetisch) — Doppelte Befundliste bei fehlerhaftem Profil. Ursache:

`ObfuscationEngine` setzt die Befunde bereits in den Meldungstext der `ConfigurationException` (`src/Obfuscation.Core/ObfuscationEngine.cs:86`), und `CommandContext.Run` hängt sie anschließend noch einmal einzeln an (`src/Obfuscation.Cli/CommandContext.cs:103`). Keine Auswirkung auf Ergebnis oder Rückgabewert, die Ausgabe ist nur doppelt so lang wie nötig (belegt in N-03). Vorschlag: in `CommandContext.Run` beim Fangen von `ConfigurationException` nur die Kopfzeile der Meldung ausgeben, nicht den vollständigen `Message`-Text, der die Befunde bereits enthält.

Behebung: die `foreach`-Schleife in `CommandContext.Run` (`src/Obfuskation.Cli/CommandContext.cs`) gestrichen — `ex.Message` enthält die Befundliste bereits. Bewusst **nicht** umgekehrt vorgegangen (Meldung im Kern kürzen): die Oberfläche zeigt dieselbe Ausnahme über `ex.Message` an und hätte dann gar keine Befunde mehr zu sehen; ein Kommentar an der geänderten Stelle hält das fest, damit die Schleife nicht später „ergänzt“ wird. Test:

```
CommandContextTests.Run_faengt_ConfigurationException_ab_und_liefert_ConfigurationError
r prüft den Rückgabewert; da ConsoleOutput direkt auf die Konsole schreibt, lässt sich die Ausgabe selbst nicht ohne größeren Umbau abfangen — dass die Befundliste genau einmal erscheint, ist von Hand nachvollzogen (obfuskation obfuscate -c <kaputtes-profil.json> <datei.csv>).
```

D-2 (Eigenschaft, dokumentationspflichtig) — `deobfuscate` mit falschem Profil liefert bei Datumsspalten stillschweigend falsche Werte. Ursache: `dateShift` besitzt keinen Tabelleneintrag (Abschnitt 4) und rechnet stattdessen über den profilweiten Offset zurück; mit einem falschen Profil ist dieser Offset ein anderer, das Ergebnis ein plausibles, aber falsches Datum, während alle anderen Spalten korrekt als `unbekanntesPseudonym` gemeldet werden (N-04, Rückgabewert bleibt 0). Vorschlag: keine Codeänderung nötig, da mit dem Prinzip aus Abschnitt 4 unvermeidlich — stattdessen in der Anwenderdokumentation und im Bericht selbst deutlicher machen, dass die Befundzahl bei richtigem Profil 0 sein muss (außer bei `redact` / `drop`); denkbar wäre ein eigener Warnhinweis im Bericht, sobald `dateShift`-Treffer neben mindestens einem `unbekanntesPseudonym`-Befund auftreten.

Behebung: am Prinzip selbst ändert sich nichts — das bleibt bewusst so (Abschnitt 4).

```
ObfuscationEngine.Deobfuscate ( src/Obfuskation.Core/ObfuscationEngine.cs ) meldet jetzt eine Warnung datumMitFremdemProfil, sobald mindestens ein Generator mit HasIntrinsicInverse == true getroffen hat und mindestens ein unbekanntesPseudonym-Befund vorliegt — der Bezug läuft über HasIntrinsicInverse, nicht über den Namen dateShift, weil ein Profil diesen Typ auch unter einem eigenen Namensraum führen kann. RunReport enthält dabei wie immer keinen Klartext und kein Pseudonym. Test: DeobfuscateWarningTests.Fremdes_Profil_bei_der_Rueckabbildung_warnt_vor_verschobenen_Daten und die Gegenprobe Beim_richtigen_Profil_bleibt_die_Warnung_aus.
```

D-3 (Bedienbarkeit) — Oberfläche verweist bei fehlerhafter Konfiguration auf nicht sichtbare Hinweise. Ursache: der Hinweisbereich hängt am ausgewählten Feld (`MainViewModel.RefreshIssues` befüllt `Issues`, aber die Anzeige liegt im Regelbereich, der ohne wählbares Feld leer bleibt); bei fehlerhafter Konfiguration bleibt die Feldliste leer, weil sich keine Engine aufbauen lässt (`src/Obfuskation.Gui/ViewModels/MainViewModel.cs:414`). Belegt in G-20: die Statuszeile meldet nur „Die Konfiguration ist fehlerhaft — siehe Hinweise“, während die Kommandozeile dieselben fünf Fehler mit Feldpfad nennt (N-03). Vorschlag: die `Issues`-Liste zusätzlich unabhängig vom gewählten Feld anzeigen, etwa in einem eigenen Bereich oberhalb der Feldliste, solange keine Engine aufgebaut werden kann.

Behebung: `MainViewModel` führt jetzt `_engineIsBroken` mit und darüber `public bool HasBlockingIssues => _engineIsBroken && Issues.Count > 0;`, gesetzt in `RefreshAnalysis` an der Stelle, an der bisher ohne weitere Anzeige abgebrochen wurde. `MainWindow.axaml` zeigt dafür eine eigene Karte „Hinweise zur Konfiguration“ oberhalb der Feldliste (`Grid.Row="2"` , `IsVisible="{Binding HasBlockingIssues}"`), mit demselben `ItemsControl` -Aufbau wie der bestehende, weiterhin vorhandene Hinweisbereich im Regelbereich — der bleibt für den Fall einer funktionierenden Engine der richtige Platz. Test: `MainViewModelTests` , ein Profil mit unbekanntem Generator ergibt `HasBlockingIssues == true` , ein gültiges Profil `false` .

D-4 (Fehler, zerstörend) — Generatorauswahl blieb leer bei einem eigenen Namensraum und löschte den Generator. Behoben in 1.0.1. Ursache: `GeneratorOption.Find` suchte nur in `BuiltIn` und legte sonst ein neues `GeneratorOption(name, "eigener Namensraum")` an; die Auswahlliste stammt aber aus `GeneratorOption.For` , wo derselbe Eintrag die Erklärung „eigener Namensraum, wie numericId“ trägt. `GeneratorOption` ist ein `record` und vergleicht über den Wert — die beiden Instanzen waren also nicht gleich, die ComboBox fand keinen passenden Eintrag und setzte ihren `SelectedItem` auf `null` . Der Setter von `SelectedGenerator` schrieb dieses `null` in die Regel zurück: **das bloße Ansehen des Feldes entfernte den Generator.** Wurde das Profil danach gespeichert, war er dauerhaft weg; wer ihn von Hand ergänzte und dabei `numericId` statt `belegNummer` wählte, hob die Trennung der Namensräume auf. Dieselbe Ursache traf `TextRuleViewModel.Generator` .

Behebung: `Find` nimmt jetzt das Profil entgegen und sucht in `For(profile)` ; `TextRuleViewModel` bekommt das Profil gereicht. Festgehalten durch `Ein_Feld_mit_eigenem_Namensraum_zeigt_seinen_Generator_an` und `Auch_eine_Textregel_zeigt_einen_eigenen_Namensraum_an` , die beide prüfen, dass der gewählte Eintrag in der Auswahlliste enthalten ist — genau daran scheiterte es.

D-5 (Darstellung) — Bildlaufleiste überdeckt die Anzahlen im Fenster „Ersetzungstabelle“. Die rechtsbündigen Zahlen je Namensraum werden teilweise angeschnitten. Betroffen ist vermutlich das Layout in `src/Obfuskation.Gui/Views/MappingWindow.axaml` — der Container für die Liste müsste der Bildlaufleiste Platz reservieren (etwa über einen rechten Rand oder `Padding` auf dem Inhalt statt auf dem `ScrollViewer`).

Behebung: im `DataTemplate` der Namensraum-Liste (`src/Obfuskation.Gui/Views/MappingWindow.axaml`) `Margin="0,3"` auf `Margin="0,3,14,3"` erweitert — der rechte Rand gehört an den Inhalt, nicht an den `ScrollViewer` : dessen `Padding` läge innerhalb des Sichtbereichs und verschöbe die Bildlaufleiste mit, das Problem bliebe bestehen. Nur XAML, manuell nachgeprüft.

D-6 (Darstellung) — Hinweistext im Fenster „Textregeln“ rechts abgeschnitten. Der Text „höhere Priorität gewinnt bei Überlappung“ ist unvollständig lesbar. Betroffen ist `src/Obfuskation.Gui/Views/TextRulesWindow.axaml` ; Abhilfe wäre Zeilenumbruch (`TextWrapping="Wrap"`) statt fester Breite für dieses Element.

Behebung: `TextWrapping="Wrap"` ergänzt und den `TextBlock` aus der waagerechten `StackPanel` in eine eigene Zeile des umgebenden `Grid` verschoben, damit die Umbrucherlaubnis auch eine begrenzte Breite bekommt — in der `StackPanel` wäre unendlich viel Platz zugemessen worden und der Text hätte nie umgebrochen. Nur XAML, manuell nachgeprüft.

D-7 (Sprache) — `--help` mischt Deutsch und Englisch. Teilweise behoben in 1.3.0.

Description: und „Show help and usage information“ stammen aus den Vorgaben von `System.CommandLine` (Paket `System.CommandLine 2.0.11`, `src/Obfuskation.Cli/Obfuskation.Cli.csproj:8`) und wurden nicht lokalisiert. Kosmetisch, keine Auswirkung auf die Bedienung.

Messung statt Vermutung: Per Reflection gegen die installierte Fassung 2.0.11 geprüft (`System.CommandLine.Properties.Resources`, das interne `ResourceManager`-Backing hinter `LocalizationResources`): das deutsche Ressourcenset (Kultur `de`, ausgeliefert als `system.commandline/2.0.11/lib/net8.0/de/System.CommandLine.resources.dll`) übersetzt `HelpUsageTitle`, `HelpOptionsTitle`, `HelpArgumentsTitle` und `HelpCommandsTitle` durchaus (`Nutzung:`, `Optionen:`, `Argumente:`, `Befehle:` — sichtbar in jeder Hilfeausgabe), lässt aber ausgerechnet `HelpOptionDescription` („Show help and usage information“) und `HelpDescriptionTitle` („Description:“) auf dem englischen Neutralwert stehen. Das ist eine Lücke in den Übersetzungsressourcen des Pakets selbst, keine bewusste Entscheidung von `System.CommandLine` gegen Lokalisierung.

Behoben: `HelpOptionDescription` lässt sich reparieren, weil der angezeigte Text der Hilfeoption über die ganz gewöhnliche, öffentliche Eigenschaft `Symbol.Description` läuft — dieselbe Eigenschaft, mit der auch jede selbst angelegte Option beschriftet wird.

`ProgramInfo.AddHelpFooter` (`src/Obfuskation.Cli/ProgramInfo.cs`) setzt sie beim Einsammeln der `HelpOption` jetzt zusätzlich auf „Zeigt Hilfe und Verwendungsinformationen an.“ Das ist keine Ersatzkonstruktion, sondern der vorgesehene Weg, den Text einer Option zu ändern. Test: `ProgramInfoTests` prüft am Objektmodell, dass die `HelpOption` der Wurzel diesen Text trägt.

Nicht behebbar ohne Ersatzkonstruktion: die Überschrift `Description:` kommt aus dem internen `HelpBuilder` von `System.CommandLine`, der die Kopfzeilen über eine `abstract`e, nicht instanziiierbare Klasse `LocalizationResources` bezieht. Beide Typen sind `internal`; von außerhalb der Paket-Assembly lässt sich weder eine Instanz erzeugen noch eine abgeleitete Klasse deklarieren (geprüft: der Versuch, `HelpBuilder` namentlich zu referenzieren, scheitert am Compiler mit „Der Zugriff ... ist aufgrund des Schutzgrads nicht möglich“). Es gibt keine öffentliche Einstiegsstelle, über die sich nur diese eine Kopfzeile ersetzen ließe — das ginge nur durch eine vollständig eigene Hilfeausgabe oder ein Nachbearbeiten der Konsolenausgabe, und genau das ist ausdrücklich nicht der Weg, den dieses Projekt für D-7 gehen soll (siehe `aufgaben/A7-dokumentation.md`). `--help` bleibt deshalb an dieser einen Stelle gemischtsprachig; die Fußzeile mit Fassung und Erstellern (Abschnitt „Fassung und Ersteller in der Hilfe“ oben) ist davon unberührt.

D-8 (Testhygiene) — `dotnet test` überschrieb die echte `~/ .config/obfuskation/gui.json` des Benutzers. Behoben in 1.0.1. Beobachtet: vor dem Testlauf verwies `recentProfiles` auf ein Wegwerfverzeichnis, danach auf ein anderes. Ursache: `MainViewModelTests` legt zwar ein Wegwerfverzeichnis für Profile an, erzeugte die Einstellungen aber mit `new GuiSettings()`; `MainViewModel` ruft an vier Stellen `_settings.Save()` auf, und `GuiSettings.FilePath` (`src/Obfuskation.Gui/Services/GuiSettings.cs:47`) löst statisch auf `$XDG_CONFIG_HOME/obfuskation/gui.json` auf — ohne gesetzte Variable also auf das echte Benutzerverzeichnis. Keine Datenschutzgefahr, da die Datei nur Pfade und die gewählte Ansicht speichert, keine Werte aus verarbeiteten Dateien; ein Testlauf muss trotzdem rückwirkungsfrei sein.

Behebung: `tests/Obfuskation.Gui.Tests/TestUmgebung.cs` setzt über einen `[ModuleInitializer]` `XDG_CONFIG_HOME` auf ein Wegwerfverzeichnis, bevor der erste Test angefasst wird, und räumt es beim Beenden weg. Der Modulinitialisierer gilt für alle Testklassen der Baugruppe — anders als eine Änderung im Konstruktor einer einzelnen Klasse. Festgehalten durch `Der_Testlauf_fasst_die_Einstellungen_des_Anwenders_nicht_an`.

D-9 (Fehler, Auslieferung) — „Speichern“ schlägt in der veröffentlichten Oberfläche fehl.

Behoben in 1.2.0. Der Klick auf `Speichern` bricht mit `Could not load file or assembly 'System.IO.Pipelines, Version=9.0.0.0' ab`, das Profil wird nicht geschrieben. Eingegrenzt: der nicht gebündelte Build (`dotnet .../obfuskation-gui.dll`) speichert fehlerfrei, das Kommandozeilenprogramm ebenso, und `ScanAndConfigTests.Profile_ueberstehen_das_Schreiben_und_Lesen_unveraendert` besteht. Der Fehler steckt also nicht in `ProfileStore`, sondern in der Auslieferung als Einzeldatei: das Bündel enthält `System.IO.Pipelines` 8.0.23 (über `CsvHelper` hereingezogen), verdeckt damit die Fassung der Laufzeit, und unter `RollForward=Major` auf einer neueren Laufzeit verlangt deren `System.Text.Json` die Assemblyfassung 9.0.0.0. Auf einem Rechner mit der in der README geforderten .NET-8-Laufzeit tritt der Fehler nicht auf.

Entscheidung: die Auslieferung bleibt framework-abhängig, mit der .NET-8-Laufzeit als Voraussetzung, wie in der README gefordert. `--self-contained` bleibt eine Option der Bauskripte (`build-release.sh` / `build-release.cmd`) für alle, die auf die Installation der Laufzeit verzichten wollen, wird aber nicht zur Vorgabe — kein Codeeingriff, keine Änderung an den Bauskripten, keine Runtime-Prüfung beim Programmstart.

Behebung: `System.IO.Pipelines` gehört seit .NET 3 zum Framework selbst; `src/Obfuskation.Gui/Obfuskation.Gui.csproj:39` schließt die von `CsvHelper` hereingezogene Paketfassung deshalb mit `<PackageReference Include="System.IO.Pipelines" Version="8.0.0" ExcludeAssets="runtime" />` aus der veröffentlichten Ausgabe aus, sodass die höhere Fassung der Laufzeit greift (ausführlicher Kommentar davor, Zeile 28–38, und Abschnitt 8 oben). **Offen bleibt:** die Nachprüfung mit dem Einzeldatei-Build unter Windows steht noch aus — der Fix ist am Linux-Build sowie über die Beschreibung des Mechanismus geprüft, nicht am tatsächlich unter Windows veröffentlichten Programm.

N-1 (Darstellung) — Untertitel „Tabelle: ...“ unten abgeschnitten. Behoben in 1.3.0. Beobachtet auf einer Windows-Installation (`befunde/Screenshot_2026-09-07_202233.png`): im Kopfbereich des Hauptfensters verlor die zweite Zeile unter der Überschrift „Obfuskation“ ihre untere Hälfte — die Unterlängen von `p`, `g` und `j` wurden von der darunterliegenden, undurchsichtigen „Datei“-Karte übermalt. Ursache: im Kopfbereich sind alle Zeilen `Auto`, ohne feste Höhe oder Clipping; die Kombination aus `TextTrimming="CharacterEllipsis"` und `TextWrapping="NoWrap"` löste bei kleiner Schrift (`FontSizeSmall = 11`, eingebettete Inter-Schrift) zuverlässig den knappen Trimming-Pfad des Text-Layouts aus (`src/Obfuskation.Gui/Views/MainWindow.axaml`, damals Zeilen 75–77).

Behebung: `TextWrapping="NoWrap"` entfernt (`CharacterEllipsis` unterbindet den Umbruch ohnehin) und `Margin="0,3,0,0"` durch `Padding="0,3,0,3"` ersetzt — `Padding` zählt in die Messung des Elements hinein, `Margin` wirkt nur außen, der untere Rand verschafft den Unterlängen damit verlässlich Platz. Ein Kommentar an der Stelle hält den Grund fest. **Herkunft und Grenze der Prüfung:** der Screenshot stammt von Windows, die Entwicklung läuft unter Linux; der Fix gilt als plausibel, nicht als am Original bestätigt, bis er unter Windows mit einem Profil nachgeprüft wurde, dessen Mapping-Pfad ähnlich lang ist wie im Screenshot. Nur XAML, keine Tests vorgesehen.

N-2 (Bedienbarkeit) — Feldinhalt gehört über die Aktionswahl. Behoben in 1.3.0. Originalwortlaut (`befunde/prompt.md`): „Wenn ein Feld ausgewählt wird, sollte oberhalb der Aktion der aktuelle Feldinhalt stehen. Erst wenn ich sehe, was in dem Feld steht, kann ich vernünftig entscheiden, welcher Generator notwendig ist.“ Der Ist-Zustand war näher am Ziel, als es wirkte, aber an drei Stellen zu schwach: der Vorschau-Block lag *unterhalb* der Aktion/Generator-Auswahl; er erschien nur, wenn zugleich eine Vorschau möglich war (`action: pseudonymize` mit vorschaufähigem Generator) — gerade ein noch unentschiedenes Feld zeigte seinen Inhalt also gar nicht; und die Stichprobe selbst war dünn: nur CSV, nur die erste Datenzeile, fest UTF-8, eine naive Zerlegung, die an Anführungszeichen und Trennzeichen im Wert zerbrach (vormals `MainViewModel.ReadSampleValues`).

Behebung: der Vorschau-Block steht jetzt *vor* dem Aktion/Generator- `Grid` in `MainWindow.axaml`, damit die Reihenfolge der Entscheidung folgt — erst sehen, was drinsteht, dann die Behandlung wählen. Die Karte hängt an der neuen `FieldRuleViewModel.HasSampleValues` (wahr, sobald mindestens ein Beispielwert vorliegt) statt an `HasPreview`; Pfeil und Vorschauwert bleiben an `HasPreview` gebunden und fallen einfach weg, wenn es noch keine gibt — genau ein Feld auf `error` oder mit einem nicht vorschaufähigen Generator braucht die Anzeige des Inhalts am dringendsten. Die eigentliche Stichprobe zog in eine neue Klasse `FieldSampler` (`src/Obfuskation.Core/Configuration/FieldSampler.cs`, Namensraum `Obfuskation.Core.Configuration`, neben `FieldInspector`): CSV läuft über `CsvReader` mit derselben `CsvConfiguration` wie der echte Lauf (`CsvProcessor`) und über `TextFormatDetector` für Zeichensatz und Trennzeichen statt fest UTF-8, JSON über `JsonDocument` analog `FieldInspector.InspectJson`, bis zu drei nichtleere Beispielwerte je Feld, höchstens 50 gelesene Zeilen als Obergrenze gegen eine durchgängig leere Spalte. Die

Mehrfachauswahl bleibt bewusst unverändert ohne Vorschau — ein Beispielwert aus einem von zwölf gewählten Feldern ließe offen, wozu er gehört. Tests:

`tests/Obfuskation.Core.Tests/FieldSamplerTests.cs` (Trennzeichen und Anführungszeichen im Wert, Windows-1252, fünf Datenzeilen, durchgängig leere Spalte, verschachteltes JSON, Textdatei) und `tests/Obfuskation.Gui.Tests/MainViewModelTests.cs` (Beispielwerte nach dem Öffnen, `HasSampleValues == true` bei `action: error`).

Erstellt von Gregor Stübner und Claude (Anthropic).